

# CAPÍTULO 1

## INTRODUCCIÓN AL DISEÑO DE ALGORITMOS Y PROGRAMACIÓN



---

### 1.1. Lección 1

Durante esta lección analizaremos conceptos básicos sobre algoritmos: qué es un algoritmo, para qué sirve, y cuál es la relación que existe entre *algoritmo* y *programa*.

Estudiaremos los recursos iniciales de programación: variables, tipos de dato, operadores aritméticos, lógicos y relacionales, y las estructuras de control de flujo de datos que describe el *teorema de la programación estructurada*.

Veremos también cómo usar la herramienta de programación Eclipse para codificar, compilar, depurar y ejecutar nuestro primer programa.

#### 1.1.1. ¿Qué es un algoritmo?

Es un conjunto finito y ordenado de pasos o acciones cuya ejecución nos permite resolver un problema.

Dicho de otro modo: dado un determinado problema, cuáles serían las acciones y en qué orden deberíamos ejecutarlas para lograr una resolución satisfactoria.



Lección 1

Ejecutamos algoritmos para resolver los problemas cotidianos que se nos presentan día a día, independientemente de cuál sea su nivel de complejidad.

Por ejemplo: si estamos parados en el cordón de la vereda y queremos cruzar la calle, el siguiente algoritmo resolverá nuestra situación problemática:

Miramos hacia la izquierda.

**Si** no viene ningún auto **entonces**:

Miramos hacia la derecha.

**Si** no viene ningún auto **entonces**:

Cruzamos.

**Sino**:

Esperamos.

**Fin Sino**

**Sino**:

Esperamos.

**Fin Sino.**

Lo anterior es sólo una mínima parte del algoritmo que ejecutamos cada vez que tenemos que cruzar una calle.

Podría suceder que al mirar hacia la izquierda (o hacia la derecha) observemos que sí viene un auto, pero se encuentra a una distancia tal que igual podremos cruzar sin ser atropellados. O tal vez, observemos que el auto no está tan lejos, pero se aproxima a baja velocidad, así que podremos cruzar de todos modos.

La observación sobre la distancia y velocidad del auto, así como la determinación del grado de peligrosidad que esto representa para nosotros (si es que decidimos cruzar la calle) implica resolver una serie de cálculos de relativa complejidad, que realizamos mentalmente y sin darnos cuenta.

Por ejemplo: llamemos  $p$  a nuestra ubicación,  $q$  a la ubicación del auto,  $v$  a su velocidad,  $z$  a nuestra velocidad (cuán rápido caminamos) y  $d$  a la distancia que debemos recorrer para llegar al cordón de la vereda de enfrente.

Entonces, el tiempo en que el auto llegará hacia nosotros lo calculamos como:  $|p-q|/v$ , y debe ser mayor que el tiempo que nos tomará cruzar la calle, que podemos calcular como:  $|p-d|/z$ . Incluso, inconscientemente manejamos un nivel de riesgo, pues no nos gustaría sentir el viento del auto pasando a 1 milímetro de nuestro cuerpo. Así que, si llamamos  $r$  al riesgo que estamos dispuestos a asumir, sólo cruzaremos la calle si se verifica que:  $| |p-q|/v - |p-d|/z | > r$ .

Esta decisión, que tomaremos en función de los cálculos precedentes, la podemos representar algorítmicamente con el siguiente fragmento de pseudocódigo:

**Si**  $| |p-q|/v - |p-d|/z | > r$  **entonces:**

Cruzamos.

**Fin Si.**

Sin embargo, no sólo los ingenieros podemos cruzar las calles de la ciudad. Todas las personas lo hacen a diario, porque el cerebro humano tiene la capacidad de realizar estas operaciones matemáticas en una milésima de segundo.

Pero hay algo más: *cruzamos*. ¿Qué significa esto? Todos comprendemos el significado de esta expresión, porque tenemos capacidad de abstracción. Pero si nos detenemos a pensar qué implica *cruzar* veremos que también requiere una serie de acciones. Se trata de un algoritmo en sí mismo.

*Cruzar* implica:

Bajar el cordón de la vereda.

Dar pasos hasta llegar al cordón de la vereda de enfrente.

Subir el cordón de la otra vereda.

Aún hay más: *dar un paso* también es un algoritmo en sí mismo, que implica:

Levantar un pie.

Adelantarlo.

Bajarlo.

Razonando así, podemos entrar en detalle tanto como queramos. Sin embargo, esto no es necesario porque nuestra capacidad de abstracción nos permite comprender qué representa un único término y cuáles acciones están involucradas.

El desafío consiste en diseñar y describir (o programar) algoritmos para que sean ejecutados por una computadora, que no tiene nuestra capacidad de abstracción.

### 1.1.2. Lenguaje algorítmico

Para documentar un algoritmo se utiliza un lenguaje algorítmico. Existen diversos lenguajes algorítmicos, pero los más utilizados son:

1. Diagramas.
2. Pseudocódigo.
3. Lenguajes de programación.

En este curso utilizaremos los tres, pero principalmente trabajaremos con el lenguaje de programación C++.

### 1.1.3. Recursos de programación

#### 1.1.3.1. La computadora

La computadora sólo puede realizar operaciones aritméticas y lógicas, y tiene mucha memoria. Pero al no tener nuestra capacidad de abstracción tendremos que indicarle, hasta el más mínimo detalle, cómo debe hacer cada una de las cosas que queremos que haga.

#### 1.1.3.2. Operaciones aritméticas y expresiones lógicas

Una operación aritmética consiste en un cálculo matemático primario: suma, resta, multiplicación, división y módulo (o valor residual). Todos los lenguajes de programación proveen estos operadores matemáticos.

Una *expresión lógica* es un enunciado susceptible de ser verdadero o falso. “2 es mayor que 5” es una expresión lógica cuyo valor de verdad es *falso*.

Más adelante, veremos que existen operadores lógicos, que usaremos para realizar operaciones entre expresiones lógicas, obteniendo como resultado una nueva expresión lógica con su correspondiente valor de verdad.

### 1.1.3.3. Lenguaje de programación

Para que una computadora pueda entender y ejecutar un algoritmo debemos escribirlo en algún lenguaje de programación. En este curso utilizaremos el lenguaje C++.

Los lenguajes de programación se componen de un conjunto de palabras en inglés y un conjunto de reglas sintácticas. La acción de escribir un algoritmo en C++ (o en cualquier otro lenguaje) se llama *codificar*. El algoritmo codificado se llama *código fuente* o simplemente *programa*.

### 1.1.3.4. Programa

Un programa es un algoritmo codificado en algún lenguaje de programación. También podríamos decir que *el algoritmo es la lógica de un programa de computadora*.

Por ejemplo, el siguiente algoritmo nos permite preguntarle a una persona cómo se llama, y luego saludarla por su nombre:

Preguntar su nombre (¿Cómo te llamas?).

Escuchar y memorizar su nombre (Preparar la memoria y esperar la respuesta).

Saludarlo por su nombre ("Hola," seguido del nombre que memorizamos).

Si el pseudocódigo del algoritmo anterior lo codificamos en C++, lo convertiremos en un programa de computación:

```
int main()
{
    // mensaje para el usuario
    cout << "Ingrese su nombre: ";

    // preparamos la memoria para guardar su nombre
    string nom;

    // el usuario ingresa su nombre y lo guardamos en memoria
    cin >> nom;

    // salida del programa: un saludo para el usuario
    cout << "Hola, " << nom << endl;

    return 0;
}
```

Las líneas que comienzan con `//` (doble barra) son comentarios o acotaciones que nos ayudarán a comprender qué hace la siguiente línea de código.

Las instrucciones `cout` y `cin`, que significan *console out* y *console in*, permiten respectivamente mostrar un mensaje (salida) e ingresar un dato (entrada).

Para ingresar un dato y recordarlo posteriormente, debemos almacenarlo en la memoria de la computadora. Esto requiere que anticipadamente dispongamos de un espacio de memoria, que en este caso usaremos para guardar una cadena de caracteres: el conjunto de caracteres que componen el nombre del usuario.

La línea que dice `string nom` reserva dicho espacio de memoria identificándolo con la palabra `nom`. En adelante, dentro del programa, usaremos `nom` para tener acceso al dato que ingresó el usuario.

Por el momento no analizaremos las palabras `main` y `return`.

#### 1.1.3.5. Variables y tipos de dato

Los identificadores que utilizamos para representar espacios de memoria se llaman variables. Por ejemplo: la variable `nom` del programa anterior.

Las variables permiten guardar datos en la memoria de la computadora. El espacio de memoria que representa una variable debe estar preparado especialmente en función de cuál sea el tipo de dato que allí queremos guardar.

En el ejemplo anterior, el tipo de dato de la variable `nom` es `string`, lo que nos permite guardar valores alfanuméricos (*cadena de caracteres*).

Sin embargo, si en el programa le hubiésemos pedido al usuario que ingresara su edad, habríamos necesitado disponer de una variable tipo numérico entero: `int`. Y si le hubiéramos preguntado por su altura, probablemente, la variable debería haber sido tipo numérico real: `double`.

Observemos que llamamos `nom` a la variable, justamente porque la utilizaremos para guardar el nombre de una persona. Podríamos haberla llamado `n`, `x`, `pepe` o `theWalrusWasPaul`.

No importa cuál sea el nombre de la variable; aunque sí es muy importante escoger un nombre que describa cuál es el dato que la variable va a contener, pues eso nos ayudará a incrementar la *legibilidad del código*.

Por ejemplo, si en el algoritmo vamos a pedir ingresar: nombre, edad y altura, deberíamos declarar tres variables cuyos nombres podrían ser los siguientes:

| Opción 1                                           | Opción 2                                   | Opción 3                              | Opción 4                              |
|----------------------------------------------------|--------------------------------------------|---------------------------------------|---------------------------------------|
| <pre>string nombre; int edad; double altura;</pre> | <pre>string nom; int ed; double alt;</pre> | <pre>string n; int e; double a;</pre> | <pre>string x; int y; double z;</pre> |

Tabla 1.1. Nombres de variables

Claramente, la Opción 1 es la más apropiada, y la Opción 4 sería la peor elección. Sin embargo, funcionalmente todas son correctas.

Las variables deben comenzar con un carácter alfabético o un guion bajo ('\_'). No pueden contener símbolos de puntuación ni operadores de ningún tipo.

| Nombres correctos                                                                                               | Nombres incorrectos                                                                                  |
|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <pre>string fecha; string fecha_nacimiento; string fechaNacimiento; string fecha3; string f; string _fec;</pre> | <pre>string fecha-nacimiento; string fecha+nacimiento; string 3fecha; string fecha.nacimiento;</pre> |

Tabla 1.2. Nombres de variables correctos e incorrectos

#### 1.1.3.6. Convención de nombres de variables

Más allá de las diferentes posibilidades que mostramos en la tabla anterior, existe una convención de nombres que restringe y ordena el modo en que debemos llamar a las variables que declaramos en nuestros programas.

Según dicha convención, que aceptaremos y respetaremos a lo largo del curso, las variables deben escribirse en minúscula. Si su nombre se compone de dos o más palabras, cada inicial (excepto la primera) debe colocarse en mayúscula.

Por ejemplo:

```
int codigoPostal;  
string nombre;  
double alturaPromedio;
```

Por supuesto, lo anterior no impide que utilicemos nombres triviales como *i*, *j*, *k*, *p*, *q* y *aux* para nombrar aquellas variables que resultan poco relevantes. Pero sí debemos nombrarlas con letras minúsculas. Nunca en mayúscula.

#### 1.1.3.7. Consola

Más arriba mencionamos la *consola*. Simplemente diremos que se trata de un dispositivo de entrada/salida compuesto por el teclado físico de la computadora (entrada), y una ventana terminal (salida) a la que accedemos mediante la siguiente combinación de teclas: WIN + R → CMD (en Windows).

#### 1.1.3.8. Compilador

Para que la computadora pueda ejecutar un programa debemos convertir su código fuente en lo que llamaremos *código de máquina*; esto es: *unos y ceros*. Pues la computadora sólo comprende y ejecuta instrucciones codificadas de este modo.

Los lenguajes de programación incluyen un programa llamado *compilador* que realiza esa tarea por nosotros. Tal como se ilustra en la siguiente imagen.

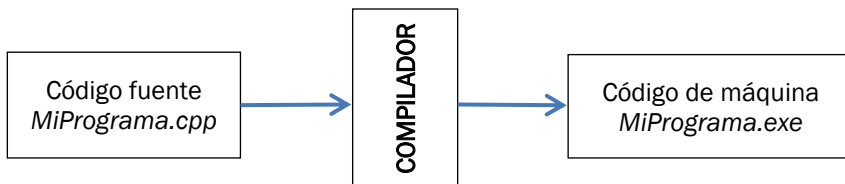


Figura 1.1. Proceso de compilación

El compilador toma el código fuente de un programa (que debe estar contenido en un archivo con extensión *.cpp*), y genera un archivo con extensión *.exe* conteniendo el código de máquina (*unos y ceros*) que sí podrá ser ejecutado por la computadora.

### 1.1.3.9. Entorno Integrado de desarrollo

El trabajo del programador consiste en diseñar el algoritmo, codificarlo, compilarlo, probarlo y depurarlo.

Existen herramientas para asistirnos durante todo el proceso de desarrollo, integrando estas tareas y facilitando el trabajo de los programadores. Se denominan *Entorno Integrado de Desarrollo* o IDE (*Integrated Development Environment*).

Las IDE pueden ser *open source*, de pago, profesionales o de estudio. La elección de cuál IDE utilizar dependerá de nuestro gusto y/o presupuesto.

Aquí utilizaremos una versión portable de Eclipse. Esta herramienta es *open source*, no tiene costo y es de uso profesional. Además, es idéntica a la versión que se utiliza para programar en Java; lo que será una ventaja para el lector cuando decida aprender dicho lenguaje de programación.



Descargar Eclipse



Instalar y usar Eclipse

### 1.1.4. Teorema de la programación estructurada

Este teorema asegura que cualquier problema computacional puede resolverse mediante un algoritmo compuesto por acciones de tres tipos diferentes.

Los tres tipos de acción a los que el teorema hace referencia son:

1. *Acción secuencial*, también llamada *acción simple*.
2. *Acción condicional*, también llamada *acción de decisión*.
3. *Acción iterativa*, también llamada *acción de repetición*.

Además, todas las acciones deben tener una única entrada y salida. Pero de esto hablaremos más adelante.

Por ejemplo, al plantear el algoritmo que resuelve el problema de cruzar la calle, usamos una acción condicional para determinar si debemos cruzarla o no.

**Si** |  $|p-q|/v - |p-d|/z$  |  $> r$  **entonces:**

Cruzamos.

**Fin Si.**

Dicho de otro modo, sólo ejecutaremos las acciones comprendidas en *cruzamos* si se verifica que  $|p-q|/v - |p-d|/z$  es mayor que  $r$ .

#### 1.1.4.1. Acción simple

Mostrar un mensaje en la consola, leer un dato por teclado, recordar un valor en la memoria o ejecutar una operación aritmética o lógica son todas acciones simples. De hecho, el programa que analizamos anteriormente está compuesto íntegramente de acciones simples.

Dado que ya vimos su codificación en C++, aquí analizaremos cómo representarlo mediante un diagrama.

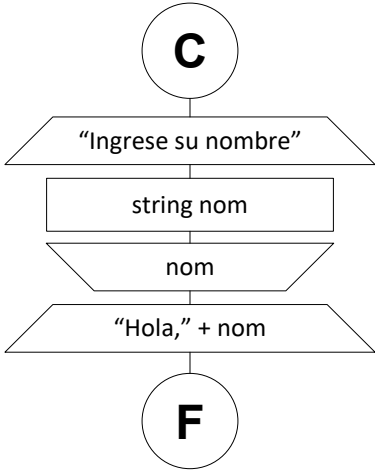
| Diagrama                                                                                                                                                                                                                                                                                            | Código C++                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <pre> graph TD     C((C)) --&gt; Input[/"Ingrese su nombre"/]     Input --&gt; Decl[rectangle: string nom]     Decl --&gt; Read[/nom/]     Read --&gt; Output[/"Hola," + nom/]     Output --&gt; F((F))   </pre> | <pre> int main() {     // mensaje para el usuario     cout &lt;&lt; "Ingrese su nombre: ";      // preparamos la memoria...     string nom;      // guardamos el nombre ingresado     cin &gt;&gt; nom;      // saludo al usuario     cout &lt;&lt; "Hola, " &lt;&lt; nom &lt;&lt; endl;      return 0; }   </pre> |

Figura 1.2. Comparación entre dos formas de representación algorítmica: diagrama y código fuente C++

El diagrama comienza con una *C* (de comienzo) y finaliza con una *F* (de fin). Luego, con un trapecio abierto hacia abajo representamos la *salida por consola*.

A continuación, en un rectángulo declaramos la variable `nom` cuyo tipo de dato es `string`, y con un trapecio abierto hacia arriba representamos el *ingreso de datos*. Finalmente, usamos una *salida* para mostrar el saludo.

Podemos apreciar que existe una relación directa entre el diagrama y el código fuente del programa.

**1.1.4.2. Acción condicional**

La acción condicional permite evaluar una expresión lógica y determinar, en función de cuál sea su valor de verdad, qué conjunto de acciones se debe ejecutar.

En el algoritmo de *cómo cruzar una calle*, usamos una acción condicional para determinar qué hacer: *cruzar* o *esperar*.

**Si** no vienen autos por la derecha **entonces:**

**Si** no vienen autos por la izquierda **entonces:**

Cruzo.

**Sino:**

Espero.

**Fin Sino.**

**Sino:**

Espero.

**Fin Sino.**

En el pseudocódigo que describe este algoritmo utilizamos dos acciones condicionales anidadas: si se verifica que no vienen autos por la derecha, entonces, validamos que no vengan autos por la izquierda. Si también se verifica esta segunda condición, entonces, *cruzamos*. En cualquier otro caso: *esperamos*.

Lo anterior podría simplificarse usando el operador lógico Y (*and*), así:

**Si** no vienen autos por la derecha **Y** no vienen autos por la izquierda **entonces:**

Cruzo.

**Sino:**

Espero.

**Fin Sino.**

Veamos otro ejemplo que requiere usar una acción condicional: un programa para calcular la división entre dos valores numéricos enteros que ingresará el

usuario. Debemos validar que el divisor sea diferente de cero, ya que la división por cero no está determinada.

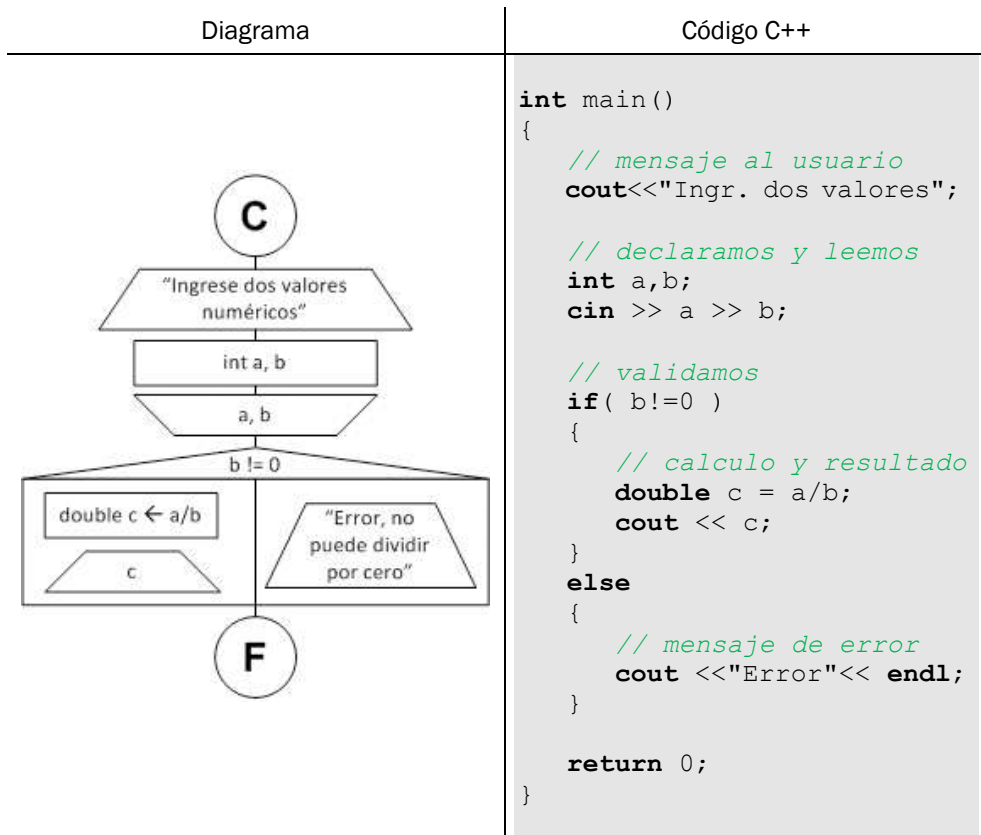


Figura 1.3. Acción condicional: representación mediante diagrama y codificación

En C++, el operador `!=` significa *distinto (not equals)*. Luego, `b!=0` debe leerse como *b es distinto de cero*. Esto puede resultar verdadero o falso dependiendo de qué valor haya ingresado el usuario.

La acción condicional se codifica con la palabra `if`. Y tiene dos partes: `if` y `else`. Si la expresión lógica `b!=0` resulta verdadera se ejecutarán las acciones que están encerradas dentro de las llaves del `if`. De lo contrario, se ejecutarán las acciones encerradas dentro de las llaves del `else`.

En el diagrama, representamos la acción condicional como un triángulo apoyado sobre dos rectángulos. En el triángulo, escribiremos la expresión lógica cuyo

valor de verdad debe ser evaluado. Si dicho valor resulta verdadero se ejecutarán las acciones ubicadas en el rectángulo de la izquierda. De lo contrario, se ejecutarán las acciones colocadas en el rectángulo de la derecha.

Cabe aclarar que sólo se ejecutará uno de los dos conjuntos de acciones. El de la izquierda (el `if`) o el de la derecha (el `else`). Nunca los dos.

En este caso, el algoritmo que resuelve el problema se compone de varias acciones simples y una acción condicional. Incluso, dentro del `if` hay dos acciones simples. Dentro del `else` hay una única acción simple.

Analicemos otro ejemplo que requiere acciones condicionales. Se trata de una calculadora muy sencilla, donde el usuario ingresará dos valores (operandos) y un operador aritmético; el programa le mostrará el resultado de la operación.

Observemos el código del programa, luego lo analizaremos.

```
int main()
{
    int a,b; // operandos
    char op; // operador (tipo char: contiene un caracter)
    double resultado;

    int error = 0;

    // leemos los datos
    cout << "Ingrese la expresion: ";
    cin >> a >> op >> b;

    if( op=='+' )// determinamos cual es el operador
    {
        resultado = a+b;
    }
    else
    {
        if( op=='-' )
        {
            resultado = a-b;
        }
        else
        {
            if( op=='*' )
            {
```

```

        resultado = a*b;
    }
    else
    {
        if( op=='/' )
        {
            if( b!=0 )
            {
                resultado = a/b;
            }
            else
            {
                cout << "ERROR: Divide por cero" << endl;
                error = 1;
            }
        }
        else
        {
            cout << "ERROR: Operador incorrecto" << endl;
            error =1;
        }
    }
}

// si no hubo error podemos mostrar el resultado
if( error==0 )
{
    cout<< a << op << b << "=" << resultado << endl;
}

return 0;
}

```

El programa es bastante simple, aunque el hecho de anidar acciones condicionales (un `if` dentro de otro `if`) lo puede tornar algo engorroso.

La lógica es la siguiente: comenzamos leyendo una expresión compuesta por un operando (`a`), un operador (`op`) y otro operando (`b`). Luego determinamos cuál es el operador en cuestión. Para esto utilizamos un `if`, para preguntar si se trata de la suma; en tal caso asignamos `a+b` a la variable `resultado`.

Si el operador no fue `+`, entonces (por `else`) preguntamos si fue `-` (resta). De ser así asignamos `a-b` a la variable `resultado`. Continuamos el proceso para determinar si el operador fue `*` (producto) o `/` (división). En este último caso, debemos verificar que `b` (el segundo operando) no sea 0 (cero), ya que la división por cero no está determinada. Si no hubo error mostramos el resultado de la operación.

Utilizamos la variable `error` para contener sólo dos valores: 0 para indicar que no hubo error y 1 para reflejar sí lo hubo. Podríamos interpretar que 0 es *falso* (`false`) y 1 es *verdadero* (`true`). Más adelante veremos que existe el tipo de dato `bool` preparado, justamente, para contener estos dos valores posibles.

#### 1.1.4.3. Acción iterativa

La acción iterativa permite repetir un conjunto de acciones mientras se verifique una determinada expresión lógica.

Volviendo al problema de *cruzar la calle*, habíamos llegado a la conclusión de que *cruzar* implicaba *bajar el cordón, dar pasos hasta llegar al cordón de la vereda de enfrente y subir el cordón*.

Podríamos reformular *dar pasos hasta llegar al cordón de la vereda de enfrente* por: *mientras no lleguemos al cordón de la vereda de enfrente, dar pasos*. Así, el pseudocódigo del algoritmo lucirá del siguiente modo:

Bajar el cordón.

**Mientras** no llegue al cordón de la vereda de enfrente **hacer**:

Dar un paso.

**Fin Mientras.**

Subir el cordón.

Como ejemplo de uso de la acción iterativa, analizaremos un programa que muestra por consola los primeros  $n$  números naturales (incluyendo el cero), donde  $n$  es un valor que ingresará el usuario.

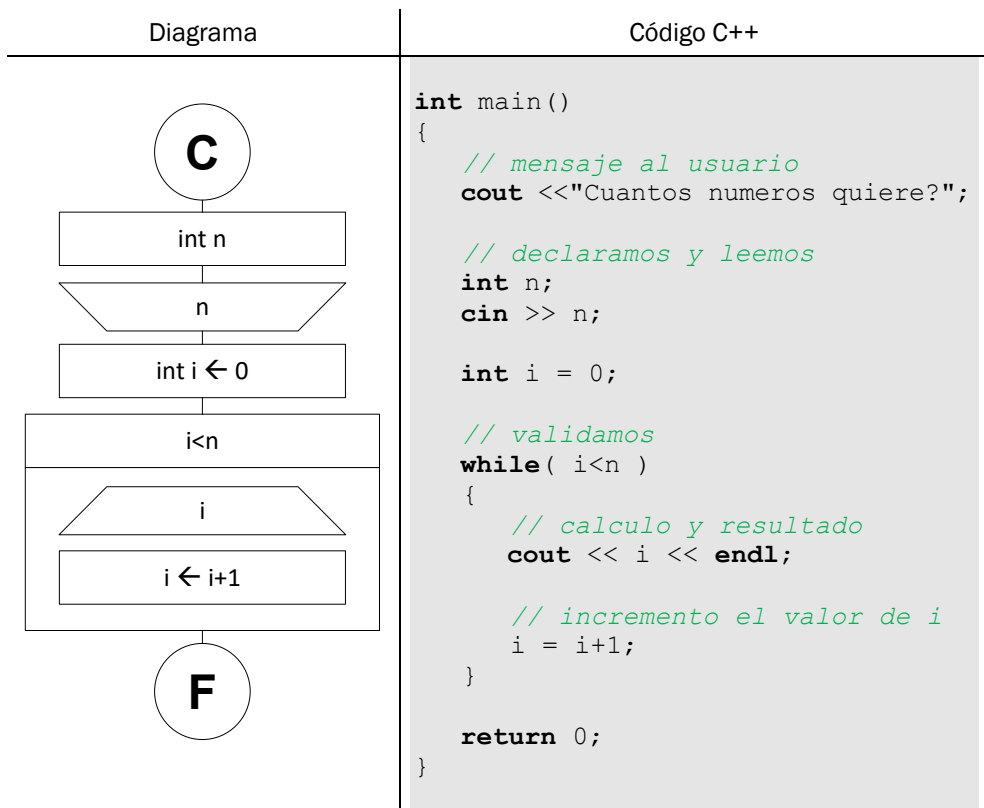


Figura 1.4. Acción iterativa: representación mediante diagrama y codificación

En los diagramas omitiremos poner los mensajes (salidas) destinados al usuario, pues su objetivo es reflejar la lógica del algoritmo. La amigabilidad del programa no forma parte de su lógica.

La acción iterativa se representa con dos rectángulos: el rectángulo superior es una especie de cabecera que indica cuál es la condición que se debe cumplir para que el ciclo itere una vez más. El rectángulo principal es el contenedor; todas las acciones que coloquemos allí dentro se repetirán una y otra vez, mientras que se verifique la expresión lógica indicada en la cabecera.

En C++, la acción iterativa se codifica con la palabra `while`.

La estrategia para resolver el problema consiste en utilizar una variable que vaya adquiriendo, uno a uno, los valores que vamos a mostrar: 0, 1, 2, ...,  $n-2$ ,  $n-1$ .

Para esto declaramos la variable  $i$  y le asignamos el valor inicial 0 (cero). Luego, todo lo que hagamos se repetirá exactamente  $n$  veces (ya veremos cómo) por estar ubicado dentro del ciclo iterativo.

*Primera iteración:* Mostramos el valor de  $i$  (que es 0) y le sumamos 1, dejándolo en 1.

*Segunda iteración:* Mostramos el valor de  $i$  (que es 1) y le sumamos 1, dejándolo en 2.

*Tercera iteración:* Mostramos el valor de  $i$  (que es 2) y le sumamos 1, dejándolo en 3.

El problema está resuelto, aunque falta hacer que el ciclo iterativo se detenga. Para esto escogeremos la condición:  $i < n$ . Es decir: mostraremos el valor de  $i$  y lo incrementaremos mientras *que* su valor se mantenga menor que  $n$ .

Cabe destacar que, dentro del ciclo iterativo, modificamos el valor de  $i$ : lo incrementamos. De este modo, en algún momento  $i$  alcanzará el valor de  $n$ . Esto hará que la condición del ciclo se deje de cumplir, poniendo fin a las iteraciones.

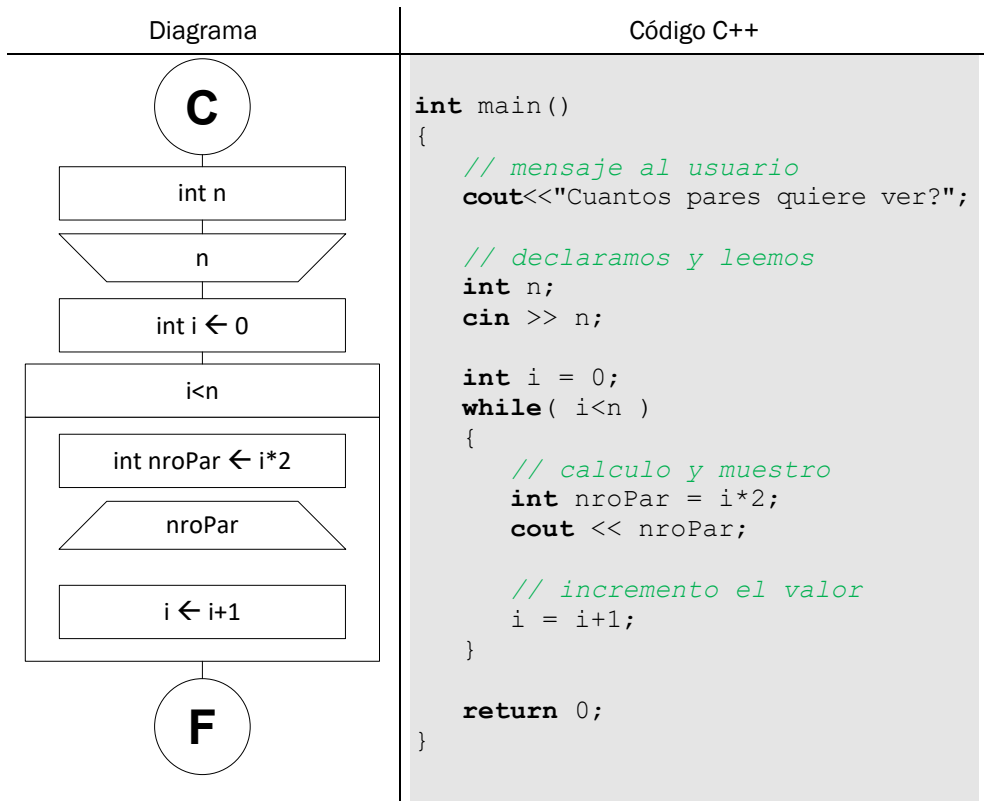
Diremos que la expresión  $i=i+1$  se llama *contador*. De hecho, estamos usando la variable  $i$  para contar cuántas iteraciones lleva realizando el ciclo iterativo, que además coincide con la cantidad de números naturales que mostraremos.

La figura 1.5. ilustra otro ejemplo: un algoritmo que muestra por consola los primeros  $n$  números pares, donde  $n$  es un valor numérico que ingresará el usuario.

Comenzamos declarando la variable  $n$  y leyendo cuántos números pares quiere ver el usuario. Luego declaramos la variable  $i$ , le asignamos el valor inicial 0 (cero), e ingresamos a un ciclo de repeticiones que iterará mientras que el valor de  $i$  sea menor que  $n$ .

Dentro de este ciclo, durante la primera iteración el valor de  $i$  será 0; en la segunda iteración será 1; en la tercera iteración será 2; y así sucesivamente.

Para obtener el número par que corresponde a la  $i$ -ésima iteración simplemente debemos multiplicar  $i*2$  y asignarlo a la variable `nroPar`.

Figura 1.5. Ejemplo: muestra los primeros  $n$  números pares

Sea cual fuere el valor de  $i$ , al multiplicarlo por 2 obtendremos el siguiente número par, como podemos apreciar en siguiente la tabla.

| i | i*2 |
|---|-----|
| 0 | 0   |
| 1 | 2   |
| 2 | 4   |
| 3 | 6   |
| 4 | 8   |
| 5 | 10  |
| : | :   |

Tabla 1.3. Números pares

Veamos otro ejemplo: cómo calcular el producto entre dos valores que ingresará el usuario. Pero no utilizaremos el operador `*`, lo haremos mediante sumas sucesivas. Es decir:  $a*b$  se calcula sumando:  $a$  veces  $b$  o  $b$  veces  $a$ .

La lógica del programa es la siguiente: leemos  $a$  y  $b$ , y utilizamos una estructura de repetición para iterar exactamente  $a$  veces. Por cada iteración sumamos el valor de  $b$  a la variable `sum`, cuyo valor inicial es 0 (cero). De este modo, estaremos sumando (acumulando) sobre dicha variable el valor de  $b$ ,  $a$  veces.

```
int main()
{
    // leemos a y b
    int a,b;
    cout << "Ingrese a y b: ";
    cin >> a >> b;

    // aqui llevaremos la suma parcial
    int sum=0;

    // iteramos a veces
    int i=0;
    while( i<a )
    {
        sum = sum+b; // sumamos a veces b
        i = i+1;
    }

    // salida del programa
    cout << a << "*" << b << "=" << sum << endl;

    return 0;
}
```

#### 1.1.4.4. Acciones de única entrada y única salida

Los tres tipos de acción que describe el teorema de la programación estructurada deben tener una única entrada y salida. Esto significa que el flujo de ejecución del algoritmo siempre será de arriba hacia abajo; y en ningún caso se podrá omitir la ejecución de alguna/s instrucción/es.

Por esto, se prohíbe el uso de la instrucción `goto`; la cual no explicaremos ni volveremos a mencionar.

### 1.1.5. Más recursos de programación

#### 1.1.5.1. Contadores y acumuladores

En el programa anterior, utilizamos la expresión `i=i+1` para incrementar en una unidad el valor de la variable `i`, por cada iteración del ciclo `while`. Diremos que `i`, utilizada de este modo, es un *contador*. Pues su función, en este caso, es contar cuántas iteraciones lleva realizadas la estructura de repetición.

La expresión `sum=sum+b`, que utilizamos en el programa anterior, permite acumular el valor de `b` en la variable `sum`. Diremos que `sum` es un *acumulador*.

#### 1.1.5.2. Prueba de escritorio

Una prueba de escritorio es una herramienta analítica que nos ayuda a determinar si el algoritmo que diseñamos funciona correctamente. Pues nos permite visualizar qué valores adquieren las variables que utilizamos en el programa.

Consiste en un seguimiento, paso a paso, de cada una de las acciones que ejecuta el algoritmo, tomando nota de qué valores asignamos a las variables.

Realizaremos una prueba de escritorio para el algoritmo del apartado anterior, que calcula el producto entre dos valores mediante sumas sucesivas.

Debemos respetar los siguientes pasos:

1. *Identificar cuáles son las variables independientes del algoritmo.* Es decir, aquellas variables cuyo valor inicial es arbitrario; generalmente las variables independientes contienen los valores ingresados por el usuario. En nuestro caso, `a` y `b` son variables independientes.
2. *Dibujar una tabla compuesta por filas y columnas.* Cada columna representa una variable del algoritmo, comenzando por las variables independientes; luego todas las demás, en el orden en que aparecen en el programa.
  - a. Si el algoritmo contiene una estructura iterativa, podemos incluir una columna para representar la condición lógica de dicho ciclo.
  - b. Cada fila de la tabla representará una iteración.

La tabla de apoyo que nos permitirá realizar la prueba de escritorio para nuestro algoritmo la veremos a continuación.

Las columnas representan a las variables  $a$ ,  $b$  (que son variables independientes),  $sum$ ,  $i$  y una columna adicional para representar la condición del ciclo iterativo ( $i < a$ ).

El video apuntado por el QR de la derecha muestra cómo completar la tabla para realizar la prueba de escritorio, y así poder determinar si el algoritmo funciona correctamente.



*Prueba de escritorio*

| Variables      |     |              |     |         |
|----------------|-----|--------------|-----|---------|
| Independientes |     | Dependientes |     |         |
| $a$            | $b$ | $sum$        | $i$ | $i < a$ |
|                |     |              |     |         |
|                |     |              |     |         |
|                |     |              |     |         |
|                |     |              |     |         |
|                |     |              |     |         |

*Tabla 1.4. Prueba de escritorio*

### 1.1.5.3. Operadores aritméticos

Los lenguajes de programación proveen operadores a través de los cuales podemos realizar operaciones aritméticas. La tabla 1.5. resume los operadores aritméticos disponibles en C++.

El operador  $\%$  (módulo o resto) retorna el residuo que se origina al dividir sus operandos. Por esto  $7\%3$  es 1, ya que  $7/3$  es 2 y resta 1.

Otro ejemplo:  $15\%11$  es 4, pues 11 entra 1 vez en 15 y sobran 4. Y  $5\%8$  es 5, ya que 8 no entra ni una sola vez en 5; sobran 5.

Por otra parte, el operador  $/$  (división) realiza la división entre sus operandos y retorna el cociente. Pero si el tipo de dato de ambos operandos es `int` entonces obtendremos, como resultado de la división, un valor entero, tipo `int`.

| Operación      | Operador | Ejemplo                                                       | Explicación                                                                    |
|----------------|----------|---------------------------------------------------------------|--------------------------------------------------------------------------------|
| Suma           | +        | <pre>int a,b; cin&gt;&gt; a &gt;&gt; b; int c = a+b;</pre>    | Si el usuario ingresa 2 y 3, luego de asignar $c=a+b$ el valor de $c$ será 5.  |
| Resta          | -        | <pre>int a,b; cin&gt;&gt; a &gt;&gt; b; int c = a-b;</pre>    | Si el usuario ingresa 5 y 3, luego de asignar $c=a-b$ el valor de $c$ será 2.  |
| Multiplicación | *        | <pre>int a,b; cin&gt;&gt; a &gt;&gt; b; int c = a*b;</pre>    | Si el usuario ingresa 2 y 3, luego de asignar $c=a*b$ el valor de $c$ será 6.  |
| División       | /        | <pre>int a,b; cin&gt;&gt; a &gt;&gt; b; double c = a/b;</pre> | Si el usuario ingresa 6 y 2, luego de asignar $c=a/b$ el valor de $c$ será 3.  |
| Módulo         | %        | <pre>int a,b; cin&gt;&gt; a &gt;&gt; b; int c = a%b;</pre>    | Si el usuario ingresa 7 y 3, luego de asignar $c=a\%b$ el valor de $c$ será 1. |

Tabla 1.5. Operadores aritméticos disponibles en C++

En el ejemplo del operador de división, aceptamos que el usuario ingresa los valores 6 y 2. El cociente entre ambos operandos es 3. Pero si el usuario ingresa 6 y 4 en las variables  $a$  y  $b$  respectivamente, siendo ambas de tipo `int`, el resultado será 1; pues 4 entra una única vez en 6.

Para obtener la división real, el tipo de dato de alguno de los operandos que intervienen en la operación debe ser `double`. Podemos convertir temporalmente el tipo de dato de una variable anteponiéndole (entre paréntesis) el tipo de dato provisorio que le queremos dar.

```
int a,b;
cin>> a >> b;
double c = (double)a/b; // casteamos
```

Esta conversión de tipo de dato se llama *cast* y su alcance es temporal. Sólo tiene efecto en la línea donde fue colocada; luego, la variable *a* seguirá siendo *int*.

De igual modo, podríamos haber *casteado* la variable *b* obteniendo exactamente el mismo resultado en la división.

```
double c = a/(double)b; // casteamos
```

#### 1.1.5.4. Otras operaciones matemáticas

Cuando hablamos de operadores aritméticos nos referimos a los operadores primitivos: *+*, *-*, *\**, */* y *%*. Pero el lenguaje de programación permite realizar otras operaciones matemáticas, como ser: *potencia*, *raíz cuadrada*, operaciones trigonométricas, *logaritmos*, *valor absoluto*, etcétera.

Estas operaciones no son primitivas, lo cual significa que están programadas en C++ y son provistas como parte de la biblioteca de funciones del lenguaje.

Para incluir las bibliotecas que nos darán acceso a estas y otras funciones utilizaremos la instrucción `#include`.

En el siguiente programa, incluimos las bibliotecas `math.h` y `stdlib.h` e invocamos las funciones `abs`, `sqrt` y `log` (valor absoluto, raíz cuadrada y logaritmo natural respectivamente).

```
// incluimos algunas bibliotecas
#include<math.h>
#include <stdlib.h>

int main()
{
    int a = abs(-1);    // valor absoluto
    cout << a << endl; // salida: 1

    int b = sqrt(64);   // raiz cuadrada
    cout << b << endl; // salida: 8

    double c = log(2.71); // logaritmo natural
    cout << c << endl;    // salida: 0.996... (casi 1)

    return 0;
}
```

### 1.1.5.5. Operadores y expresiones lógicas

Como ya hemos explicado, una expresión lógica es un enunciado susceptible de ser verdadero o falso. El valor de las siguientes expresiones lógicas es *verdadero*: “John Lennon integró Los Beatles”, “Einstein formuló la teoría de la relatividad” y “Dos es menor que cinco”.

En cambio, “Mick Jagger brilló como piloto de Fórmula 1”, “Leonardo da Vinci nació en Argentina” y “Ocho es un número impar” son expresiones lógicas cuyo valor de verdad es *falso*.

Los operadores lógicos permiten combinar expresiones lógicas para obtener una nueva expresión lógica, cuyo valor de verdad dependerá de los valores de verdad de las expresiones que la componen.

Por ejemplo: “Paul McCartney integró Los Beatles” Y “John Lennon integró Los Beatles” son dos expresiones lógicas verdaderas. Al combinarlas con el operador lógico Y (*and*) la expresión lógica resultante también resulta verdadera.

En cambio “Paul McCartney integró Los Beatles” Y “John Lennon fue un enano de Blancanieves” tienen, respectivamente, valores de verdad *verdadero* y *falso*. Al combinarlas con el operador lógico Y el resultado será una expresión lógica falsa.

El operador O (*or*) funciona diferente. La expresión “Paul McCartney integró Los Beatles” O “John Lennon fue un enano de Blancanieves” es verdadera, ya que al menos una parte de la expresión efectivamente lo es. Por último, el operador NO (*not*) niega el valor de verdad de una expresión lógica, obteniendo como resultado el valor opuesto.

El operador lógico Y será verdadero si ambos operandos lo son. El operador O resulta verdadero si al menos uno de sus operandos lo es. Teniendo en cuenta esto, podemos formular las siguientes *tablas de verdad* para dos proposiciones *p* y *q*.

| <i>p</i> | <i>q</i> | <i>p and q</i> | <i>p</i> | <i>q</i> | <i>p or q</i> | <i>p</i> | <i>not p</i> |
|----------|----------|----------------|----------|----------|---------------|----------|--------------|
| v        | v        | v              | v        | v        | v             | v        | f            |
| v        | f        | f              | v        | f        | v             | f        | v            |
| f        | v        | f              | f        | v        | v             |          |              |
| f        | f        | f              | f        | f        | f             |          |              |

Tabla 1.6. Tablas de verdad

En C++ estos operadores se codifican del siguiente modo:

|            |                               |
|------------|-------------------------------|
| <i>and</i> | <code>&amp;&amp; o and</code> |
| <i>or</i>  | <code>   o or</code>          |
| <i>not</i> | <code>!</code>                |

Tabla 1.7. Operadores lógicos

Únicamente, existen dos valores lógicos posibles: `true` (verdadero) y `false` (falso), y pueden ser contenidos en variables tipo `bool`.

```
// el usuario ingresa cuatro valores enteros
int a,b,c,d;
cin >> a >> b >> c >> d;

// ejemplo usando &&: Si a es mayor que b Y c es mayor que d
if( a>b && c>d ){ /* ... */ }

// ejemplo usando ||
if( a>b || c>d ){ /* ... */ }

bool x = a>b && c>d;
bool y = a>b && c>d;
bool z = !( !(a>b) || !(c>d) );
```

#### 1.1.5.6. Operadores relacionales

Los operadores relacionales permiten establecer si existe una relación de precedencia o igualdad entre sus operandos. Los mismos son: `>` (mayor), `<` (menor), `>=` (mayor o igual), `<=` (menor o igual), `==` (igual) y `!=` (distinto o *not equals*).

| Operación     | Operador           | Ejemplo                                                          | Explicación                                                                                                              |
|---------------|--------------------|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Mayor         | <code>&gt;</code>  | <pre>int a,b; cin &gt;&gt; a &gt;&gt; b; bool c = a&gt;b;</pre>  | Si el usuario ingresa 2 y 3, luego de asignar <code>c=a&gt;b</code> el valor de <code>c</code> será <code>false</code> . |
| Mayor o igual | <code>&gt;=</code> | <pre>int a,b; cin &gt;&gt; a &gt;&gt; b; bool c = a&gt;=b;</pre> | Si el usuario ingresa 5 y 3, luego de asignar <code>c=a&gt;=b</code> el valor de <code>c</code> será <code>true</code> . |

| Operación     | Operador | Ejemplo                                                          | Explicación                                                                                                               |
|---------------|----------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Menor         | <        | <pre>int a,b; cin &gt;&gt; a &gt;&gt; b; bool c = a&lt;b;</pre>  | Si el usuario ingresa 2 y 3, luego de asignar <code>c=a&lt;b</code> el valor de <code>c</code> será <code>true</code> .   |
| Menor o igual | <=       | <pre>int a,b; cin &gt;&gt; a &gt;&gt; b; bool c = a&lt;=b;</pre> | Si el usuario ingresa 6 y 2, luego de asignar <code>c=a&lt;=b</code> el valor de <code>c</code> será <code>false</code> . |
| Igual         | ==       | <pre>int a,b; cin &gt;&gt; a &gt;&gt; b; bool c = a==b;</pre>    | Si el usuario ingresa 7 y 3, luego de asignar <code>c=a==b</code> el valor de <code>c</code> será <code>false</code> .    |
| Distinto      | !=       | <pre>int a,b; cin&gt;&gt; a &gt;&gt; b; bool c = a!=b;</pre>     | Si el usuario ingresa 7 y 3, luego de asignar <code>c = a!=b</code> el valor de <code>c</code> será <code>true</code> .   |

Tabla 1.8. Operadores relacionales

#### 1.1.5.7. Operadores relacionales y cadenas de caracteres

Los operadores relacionales también funcionan si los utilizamos para comparar a cadenas de caracteres. En tal caso, el criterio de precedencia será alfabético.

En el siguiente programa, el usuario ingresa dos cadenas. Si la primera es alfabéticamente menor que la segunda, mostramos un mensaje en la consola.

Suponiendo que el usuario ingresa “Juan” y “Pablo”, el programa arrojará la siguiente salida: “Juan es alfabéticamente menor que Pablo”.

```
string a,b;
cin>> a >> b; // el usuario ingresa dos cadenas

if( a<b )
{
    cout << a << " es alfabeticamente menor que " << b;
}
```

### 1.1.6. Análisis de ejercicios y problemas

Los algoritmos resuelven problemas. Por esta razón, el primer paso para diseñar un algoritmo es entender cuál es el problema que el algoritmo debe resolver.

La comprensión del problema es tanto o más importante que el diseño del algoritmo que lo resuelve. Pues, difícilmente podemos diseñar un algoritmo que resuelva un problema que no llegamos a comprender en profundidad.

Es fundamental adquirir una metodología cuyas técnicas nos ayuden a transitar el proceso de entendimiento y comprensión del problema por resolver.

Aquí no propondremos ninguna metodología en particular. Pero, más allá de cuál sea la metodología que adoptemos, sus fases siempre serán las siguientes:

1. Identificar los registros de entrada, datos de contexto y salidas.
2. Entender cuáles son los procesos de transformación que, a partir de los registros de entrada y contexto, permiten generar los datos de salida.

Tomaremos como ejemplo el enunciado del siguiente problema:

*Enunciado:* Se ingresan tres valores numéricos que corresponden al día, mes y año de una fecha. Se debe validar que la misma sea consistente. Esto significa que la fecha ingresada no debería ser 35 de enero o 29 de febrero de un año que no es bisiesto. Se pide mostrar por consola un mensaje indicando si la fecha es correcta o no.

#### 1.1.6.1. Datos de entrada, de contexto y de salida

Para el problema planteado, podemos identificar los siguientes datos de entrada, contexto, y salida.

*Datos de entrada:* son los datos que ingresará el usuario:

- *día* – día del mes.
- *mes* – mes del año.
- *año* – año.

*Datos de contexto:* son parte del contexto:

- Los días y meses se numeran a partir de 1.
- Cuántos días tiene cada mes (por ejemplo: enero tiene 31 días).
- Cuántos meses tiene cada año (12 meses).

- Febrero tiene 28 o 29 días, dependiendo de que el año sea o no bisiesto.
- Un año es bisiesto si es divisible por 4, o por 400 pero no por 100. Aunque el cálculo es algo más complejo que eso, aquí lo aceptaremos así.

*Datos de salida:* qué le mostraremos al usuario. Según el enunciado mostraremos un mensaje indicando si los datos ingresados componen una fecha válida o no.

### 1.1.6.2. Procesos para transformar la entrada en salida

Los procesos de transformación son las acciones que programaremos. Qué cálculos y decisiones debemos tomar para obtener la respuesta que espera el usuario.

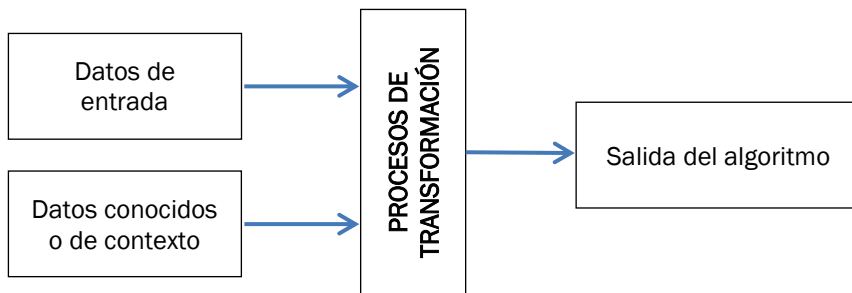


Figura 1.6. Proceso de transformación de entrada en salida

En el caso del problema de validar la fecha ingresada por el usuario, los procesos de transformación serán los siguientes:

*Validación de la fecha:* será válida si el día, mes y año son válidos.

*Validación del día:* el día (dato de entrada) debe ser mayor que cero, y menor o igual que la cantidad de días que tiene el mes (dato de entrada). Cuántos días tiene el mes es un dato de contexto.

*Validación del mes:* el mes (dato de entrada) debe ser mayor que cero, y menor o igual a 12 (dato de contexto).

*Validación del año:* el año (dato de entrada) debe ser un número entero positivo (dato de contexto).

### 1.1.7. Tipos de problema

Dependiendo de cómo se proporcionan los datos o registros de entrada y qué salida queremos obtener, los problemas se pueden clasificar en diferentes tipos. Identi-

ficar el tipo de problema es muy importante, pues problemas similares requieren soluciones similares.

#### **1.1.7.1. Problemas de registro simple y problemas de múltiples registros**

Los problemas de registro simple procesan un único juego de datos, también llamado *registro*. Por ejemplo, los siguientes enunciados describen problemas de este tipo:

- 1.1. Se ingresa un valor numérico, determinar si se trata de un número primo.
- 1.2. Se ingresan tres valores numéricos diferentes entre sí, se pide determinar cuál es el menor, medio y mayor valor.
- 1.3. Se ingresan los nombres y fechas de nacimiento de dos personas, se pide indicar el nombre de la persona más joven.

En el problema 1.1, el juego de datos de entrada (o registro) consiste en un único valor numérico  $n$ . {  $n$  }.

El problema 1.2 tiene un único registro de entrada compuesto por tres valores numéricos: {  $a, b, c$  }; una terna.

En el problema 1.3, el registro de entrada se compone en un par de valores; cada uno compuesto por el nombre de una persona y una terna que describe su fecha de nacimiento:

{ [  $nom1, (d1, m1, a1)$  ], [  $nom2, (d2, m2, a2)$  ] }

Reformularemos estos enunciados para contrastar la diferencia entre un problema de registro simple y otro de múltiples registros.

- 2.1. Se ingresan 100 valores numéricos. Por cada uno se pide determinar si se trata de un número primo.
- 2.2. Se ingresa un valor  $n$ . Luego se ingresarán  $n$  ternas de valores numéricos diferentes entre sí. Por cada una de estas ternas se pide determinar cuál es el menor, medio y mayor valor.
- 2.3. Se ingresan varios pares de valores, cada uno compuesto por el nombre de una persona y tres enteros que indican su fecha de nacimiento. Se pide indicar, por cada par, el nombre de la persona más joven.

Formulados así, los problemas 2.1, 2.2 y 2.3 procesan múltiples veces registros del mismo tipo que los procesados por sus predecesores 1.1, 1.2 y 1.3, que lo hacían una única vez. Los resultados que se espera obtener también son múltiples.

En el problema 2.1 esperamos obtener exactamente 100 resultados; 100 mensajes indicando “El número ingresado es primo” o “El número ingresado no es primo”.

En los problemas 2.2 y 2.3 también se espera obtener múltiples resultados. Tanto como registros el usuario vaya a ingresar.

#### 1.1.7.2. Multiplicidad

Podemos ser más precisos y establecer una distinción sobre el tipo de multiplicidad que describe el enunciado del problema.

En el problema 2.1 se indica que el usuario ingresará exactamente 100 registros. Diremos que se trata de un caso de *multiplicidad fija*.

Cada vez que ejecutemos el algoritmo, el usuario ingresará exactamente 100 valores. Dichos valores no necesariamente serán los mismos una y otra vez, pero sí podemos asegurar que siempre serán 100.

El enunciado del problema 2.2 dice que se ingresarán exactamente  $n$  registros, donde  $n$  es un valor que el usuario ingresará con anterioridad. A este tipo de casos los llamaremos de *multiplicidad variable conocida*, pues cada vez que se ejecute el algoritmo, el usuario podrá determinar qué cantidad de registros desea procesar.

El problema 2.3 indica que se ingresarán múltiples registros, pero no especifica cuántos. Se trata de un caso de *multiplicidad variable desconocida*. En casos como este, se debe establecer una señal para determinar que el usuario finalizó la carga de datos. Si el enunciado no indica explícitamente cuál es esta señal, tendremos que definirla nosotros mismos.

Por ejemplo, en el problema 2.3 el usuario ingresará varios registros como este:

```
{ [nom1, (d1,m1,a1)], [nom2, (d2,m2,a2)] }
```

Como el enunciado no da precisiones sobre cuál será la marca de finalización de la carga de datos, podemos establecer que se ingresará un `nom1="ZZZ"` para indicar que dicha carga de datos finalizó.

#### 1.1.7.3. Registros y tablas

Mientras que en el problema 1.1 la entrada de datos consiste en el ingreso de un único valor numérico, la entrada de datos del problema 2.1 se compone de 100 valores numéricos.

La siguiente imagen ilustra cómo podemos representar los datos de entrada de un problema. A la izquierda, vemos el único dato de entrada del problema 1.1. A la derecha, vemos los 100 valores que el usuario ingresará en el problema 2.1, que forman una tabla compuesta por 100 filas y una única columna.

|   |    |
|---|----|
| 5 | 7  |
|   | 4  |
|   | :  |
|   | 18 |

Figura 1.7. Comparación entre un registro simple y múltiples registros

Cabe aclarar que todos estos valores son hipotéticos, ya que será el usuario quién decida qué valores va a ingresar.

Para comparar los datos de entrada de los problemas 1.2 y 2.2, diremos que un registro forma una fila, y varios registros forman una tabla compuesta por múltiples filas y columnas. Entonces, los datos de entrada de estos problemas generan la siguiente fila/tabla.

|    |    |    |    |    |   |
|----|----|----|----|----|---|
| 25 | 12 | 10 | 7  | 11 | 6 |
|    |    |    | 23 | 2  | 8 |
|    |    |    | 2  | 14 | 1 |
|    |    |    | :  | :  | : |

Figura 1.8. Entrada simple contra entrada múltiple

Finalmente, los problemas 1.3 y 2.3 tienen los siguientes registros.

|        |    |   |      |        |    |   |      |
|--------|----|---|------|--------|----|---|------|
| Martín | 23 | 1 | 1995 | Germán | 14 | 3 | 2003 |
|--------|----|---|------|--------|----|---|------|

Figura 1.9. Entrada simple

|        |    |   |      |         |    |    |      |
|--------|----|---|------|---------|----|----|------|
| Juan   | 5  | 6 | 1997 | Alberto | 3  | 12 | 1994 |
| Pablo  | 14 | 2 | 2002 | Nicolás | 29 | 8  | 2002 |
| Carlos | 23 | 1 | 1995 | Diego   | 16 | 3  | 1986 |
| :      | :  | : | :    | :       | :  | :  | :    |
| FIN    |    |   |      |         |    |    |      |

Figura 1.10. Entrada múltiple

**1.1.7.4. Problemas de procesamiento horizontal y procesamiento vertical**

Esta clasificación nos ayudará a entender si los datos de salida que debemos generar surgirán de procesar una fila, una columna o una combinación de ambas.

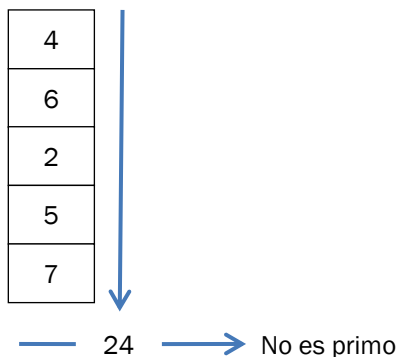
Los problemas 2.1, 2.2 y 2.3, planteados en el apartado anterior, son problemas de procesamiento horizontal, pues se espera obtener tantos resultados como filas (horizontal) sean ingresadas. Cada uno de estos resultados se obtendrá procesando exclusivamente los datos de la fila en cuestión.

Para comprender la diferencia entre ambos tipos de procesamiento, reformularemos los enunciados anteriores, de modo tal que para resolverlos debamos utilizar procesamiento vertical.

- 3.1. Se ingresan 100 valores numéricos, indicar si su suma da un número primo.
- 3.2. Se ingresan  $n$  ternas de valores numéricos diferentes entre sí, se desea calcular los promedios de los menores valores de cada terna, el promedio de los valores que se encuentran en el medio y el promedio de los mayores.
- 3.3. Se ingresan varios pares de valores, donde cada uno está compuesto por el nombre de una persona y tres enteros que indican su fecha de nacimiento. Informar el nombre de la persona más joven y el de la más vieja del conjunto.

El problema 3.1 requiere mostrar un único resultado, que surge de procesar todo el conjunto de registros. Dicho proceso consiste en sumarlos y determinar si la suma es un número primo.

Se trata claramente de un problema de procesamiento vertical.



*Figura 1.11. Procesamiento vertical*

Por cuestiones de claridad y organización, el análisis lo hemos realizado sobre un conjunto reducido de datos. Lo mismo haremos en los casos sucesivos.

En el problema 3.2 debemos procesar horizontalmente cada fila y determinar cuál es el menor, medio y mayor valor de la terna. Luego, procesaremos verticalmente estos resultados para calcular sus promedios.

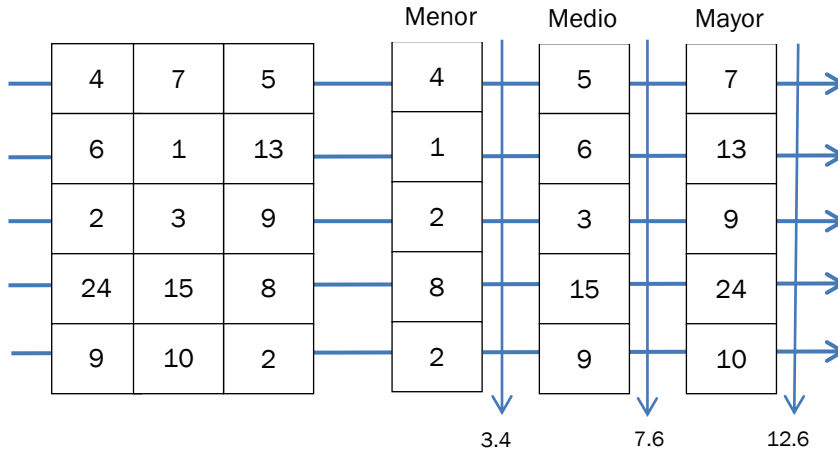


Figura 1.12. Procesamiento mixto: horizontal y vertical

Finalmente, el problema 3.3 consiste en determinar cuál es la persona más vieja y cuál la más joven de todo el conjunto de datos. La respuesta la obtendremos una vez que hayamos comparado las fechas de nacimiento de todas las filas. Se trata de un problema de procesamiento vertical.

#### 1.1.7.5. Problemas de corte de control

Los problemas de corte de control se caracterizan porque los datos de entrada se ingresan respetando un determinado ordenamiento. De este modo podremos establecer subconjuntos de registros identificados por un valor común.

Por ejemplo: se ingresan varios registros, compuestos por tres valores numéricos, que representan el legajo de un estudiante, un mes, y la cantidad de inasistencias que dicho estudiante acumuló durante el mes en cuestión.

| Legajo | Mes | Inasistencias |
|--------|-----|---------------|
| 3      | 5   | 4             |

Figura 1.13. Representación gráfica de un registro

Los datos se ingresan ordenados por legajo. Esto significa que todos los registros que corresponden a un mismo estudiante (legajo) estarán agrupados.

No se ingresarán filas para indicar que no hubo inasistencias. De este modo, por cada estudiante tendremos un registro por cada mes en el cual se registró, al menos, una inasistencia.

Se pide determinar para cada estudiante (identificado por su legajo) qué cantidad de faltas acumuló durante el año; e informar, al finalizar el proceso, el legajo del estudiante con mayor cantidad de inasistencias.

Para comprender cómo debemos procesar los datos, primero los tendremos que visualizar. Con el tiempo, esta visualización llegará a ser mental, pero inicialmente, tal como hicimos con los problemas anteriores, los graficaremos.

| Legajo | Mes | Inasistencias |              |
|--------|-----|---------------|--------------|
| 3      | 5   | 4             |              |
| 3      | 4   | 1             |              |
| 3      | 6   | 2             | Total faltas |
| 3      | 8   | 1             | 8            |
| 7      | 2   | 1             | Total faltas |
| 7      | 5   | 4             | 5            |
| 12     | 5   | 7             |              |
| 12     | 4   | 1             | Total faltas |
| 12     | 8   | 1             | 9            |
| 14     | 3   | 2             | Total faltas |
| 14     | 4   | 3             | 5            |

Figura 1.14. Ejemplo de datos de un problema de corte de control

Como vemos, los datos están ordenados por legajo. Dado que por cada legajo puede haber más de una fila, todas las filas que corresponden a un mismo estudiante se encuentran agrupadas.

El conjunto de registros está dividido en subconjuntos, cada uno de los cuales representa las inasistencias anuales de un estudiante, discriminadas mes por mes.

El enunciado no dice nada sobre un ordenamiento de segundo nivel, sólo aclara que los registros se ingresarán ordenados por legajo; pero a igual número de legajo, no necesariamente las filas estarán ordenadas por mes.

El primer resultado que debemos obtener es la sumatoria (procesamiento vertical) de las inasistencias mensuales de cada estudiante. Esto requiere sumar los valores de la columna de inasistencias, controlando que las filas procesadas corres-

pondan a un mismo legajo. Haremos esto tantas veces como subconjuntos (identificados por legajo) podamos distinguir dentro del conjunto general.

Según el ejemplo, el estudiante cuyo legajo es 3 faltó 8 veces a lo largo del año. El estudiante con legajo 7 faltó 5 veces, y así sucesivamente. Todos los demás estudiantes (aquellos que no figuran en la tabla) registraron asistencia perfecta.

Observemos que no conocemos los legajos de todos los estudiantes. Tampoco sabemos cuántos estudiantes existen. Sólo tenemos información sobre aquellos estudiantes que registraron, al menos, una inasistencia durante el año.

El problema también pide determinar cuál fue el estudiante con mayor cantidad de inasistencias. Esto requiere procesar la cantidad total de inasistencias de cada estudiante. En este caso, el resultado será único (el estudiante que más veces faltó), y estará disponible luego de que hayamos procesado todos los subconjuntos.

NOTA: Podría suceder que dos o más estudiantes registren la mayor cantidad de faltas. Por ejemplo: tres estudiantes faltaron 15 veces, y ningún otro alumno faltó tanto. Sin embargo, este tipo de situaciones no las tendremos en cuenta, porque por el momento no contamos con las herramientas necesarias para poderlas resolver.

### 1.1.8. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

## 1.2. Lección 2

Durante esta lección profundizaremos sobre *tipo de dato*, e introduciremos un concepto fundamental de programación: *funciones*; además de otros temas relacionados como argumentos, parámetros, bibliotecas y reusabilidad del código fuente.

### 1.2.1. Tipo de dato

Como ya sabemos, a través de las variables podemos guardar datos en la memoria de la computadora. Sin embargo, el espacio de memoria que representa una variable debe tener una preparación especial, que dependerá de cuál sea el tipo de dato del valor que guardaremos allí.



Lección 2

Por ejemplo, el nombre de una persona es de tipo `string` (cadena de caracteres). Lo mismo sucede con una contraseña, una descripción o una dirección de correo electrónico.

En cambio, la edad de una persona, la cantidad de cuotas en que decidimos pagar un producto, o la cantidad de materias que cursaremos en la facultad son datos de tipo numérico entero: `int`.

El saldo de una cuenta corriente, la altura de una persona o el precio de un producto son datos de tipo numérico real o flotante: `double`.

| Tipo         |        | En C++    | Longitud    |
|--------------|--------|-----------|-------------|
| Numérico     | Entero | char      | 1 byte      |
|              |        | short     | 2 bytes     |
|              |        | int       | 4 bytes     |
|              |        | long      | 4 bytes     |
|              |        | long long | 8 bytes     |
|              | Real   | float     | 4 bytes     |
|              |        | double    | 8 bytes     |
| Alfanumérico |        | string    | $n+1$ bytes |
| Lógico       |        | bool      | 1 byte      |
| Nulo         |        | void      |             |

Tabla 1.9. Tipos de dato, codificación y longitud en C++

Los lenguajes de programación establecen tipos de dato que usaremos para indicar cuál será el tipo del valor que guardaremos en el espacio de memoria representado por una variable. La tabla 1.9. muestra los diferentes tipos de dato, clasificándolos en cuatro grupos principales.

Cada tipo define una cantidad de bytes de memoria destinada a guardar el dato de la variable; y también, establece cómo deben interpretarse los *unos* y *ceros* que componen dichos bytes.

### 1.2.1.1. Tipos enteros

Según la tabla anterior, el tipo `short` (por ejemplo) representa 2 bytes de la memoria de la computadora. Dado que cada byte consiste en un conjunto de 8 bits, el valor más grande que podremos guardar en una variable tipo `short` se forma con 16 *unos* (dígitos binarios), y el más pequeño se forma con 16 *ceros*.

```
11111111 | 11111111
-----|-----
```

Como el número binario compuesto por 16 dígitos 1 equivale al número decimal  $2^{16}=65536$ , en principio, una variable `short` admite valores numéricos comprendidos entre 0 y 65535.

Sin embargo, el tipo `short`, así como todos los otros tipos enteros, está preparado para admitir valores negativos. Para esto, el bit más significativo (el que se ubica más a la izquierda) se considera *bit de signo*, y no tiene valor numérico. Sólo es un indicador del signo del número que está guardado en el espacio de memoria que representa la variable: 0 (cero) si es positivo, 1 (uno) si es negativo.

Así, el mayor valor numérico entero que podemos guardar en una variable `short` consiste en un 0 seguido de 15 dígitos 1. Esto equivale a  $2^{15}-1 = 32767$ .

```
01111111 | 11111111
-----|-----
```

El valor más negativo admitido por un `short` se compone de un 1 seguido de 15 dígitos 1. Esto es: -32767.

```
11111111 | 11111111
-----|-----
```

Como que el valor numérico 0 (cero) se considera parte de los positivos, la totalidad de las variaciones negativas representarán valores entre -1 y -32768.

En otras palabras, una variable `v` tipo `short` admite guardar valores comprendidos dentro del siguiente rango numérico:  $-32768 \leq v \leq 32767$ .

Si sabemos de antemano que en una variable tipo entero no guardaremos números negativos, podemos omitir el bit de signo utilizando la palabra `unsigned`.

```
unsigned short a;
```

Esto hará que todos sus bits, incluyendo al más significativo, sean utilizados para representar la magnitud del número. En este caso, la variable `a` admitirá valores que van desde 0 hasta 65535; que equivale a 16 dígitos 1:  $2^{16}-1$ .

El mismo análisis también es aplicable a los tipos `int`, `long` y `char`.

En general, una variable `v` cuyo tipo de dato es numérico entero, según tenga o no bit de signo, admite valores comprendidos dentro de los rangos que se indican en la siguiente tabla, donde  $n$  es la cantidad de bits representados por el tipo de dato.

| Con bit de signo                 | Sin bit de signo      |
|----------------------------------|-----------------------|
| $-2^{n-1} \leq v \leq 2^{n-1}-1$ | $0 \leq v \leq 2^n-1$ |

Tabla 1.10. Rangos numéricos admitidos por los tipos de dato entero

En la práctica, no utilizaremos los tipos `short` ni `long`, sólo usaremos `int`.

### 1.2.1.2. Tipos flotantes

Sobre los tipos reales, su estructura interna está preparada para contener una magnitud (mantisa) y un punto flotante representado por un exponente.

```

-----|-----|-----|-----
01111111|11111111|11111111|11111111
<- exp -><----- mantisa ----->
```

Como vemos, en los 4 bytes de una variable tipo `float` se representan el signo del número, un exponente y una mantisa. La mantisa es la magnitud y el exponente es la *coma o punto flotante*.

Se considera que los 8 bits del exponente colocados en 0 representan el valor -127. Es decir: la magnitud que indica la mantisa con una coma desplazada 127 lugares hacia la izquierda. Estos 8 bits colocados en 1 representan el valor 127 e indican que la mantisa lleva una coma desplazada 127 lugares hacia la derecha. El bit de signo establece si se trata de un número positivo o negativo.

El tipo `double` es similar, pero ofrece mucha mayor precisión, ya que utiliza 8 bytes de memoria. En la práctica, sólo utilizaremos el tipo `double`.

### 1.2.1.3. Tipos alfanuméricos

El tipo `string` permite guardar una cadena de caracteres, que consiste en una sucesión de caracteres individuales tipo `char`. Internamente, la cadena finaliza con un carácter especial que llamaremos `'\0'` (barra cero).

```
string s = "Pablo";
```

|   |   |   |   |   |    |
|---|---|---|---|---|----|
| P | a | b | l | o | \0 |
|---|---|---|---|---|----|

Figura 1.16. Representación interna de una cadena de caracteres

El carácter `'\0'` es automático. Por esta razón será poco frecuente que tengamos que agregarlo nosotros mismos al final de una cadena. Como este tema lo abordaremos en detalle más adelante, aquí sólo lo trataremos superficialmente.

Los caracteres tienen asociado un código numérico llamado *código ASCII*. Por ejemplo: el código ASCII del carácter 'A' es 65. El código del carácter 'B' es 66 y así sucesivamente. Por su parte, el código del carácter '0' es 48, el código del carácter '1' es 49, el código del carácter '2' es 50, etcétera.

Aunque intuitivamente podemos pensar que `char` es un tipo de dato alfanumérico, internamente se trata de un tipo numérico entero, pues las variables de este tipo guardan el número del código ASCII de un carácter.

Por lo anterior, las siguientes líneas son equivalentes.

```
char c = 'A'; | char c = 65;
```

Figura 1.17. Representación literal y numérica (ASCII) de un carácter

En el lado izquierdo, le asignamos a la variable `c` el carácter `A`, a través de su expresión literal. En el lado derecho, lo hacemos mediante su código ASCII.

Si entendemos que `'A'` es un número entero (65), no nos sorprenderemos al ver expresiones como la siguiente. En tal caso diremos que `d` vale 66 o `'B'`.

```
char d = 'A'+1;
```

Es importante comprender la diferencia entre *cadena literales*, y *caracteres literales*. Aunque podríamos pensar que se trata de lo mismo, claramente no lo es.

```
char c = 'A'; | string s = "A";
```

Figura 1.18. Comparación entre un carácter y una cadena

La variable `c` (del lado izquierdo) guarda un valor numérico entero (65), representado en 1 byte con bit de signo. En el lado derecho, la variable `s` representa 2 bytes de memoria: `"A\0"`.

#### 1.2.1.4. Tipos lógicos

Los valores lógicos, `true` (verdadero) o `false` (falso), se pueden asignar en variables tipo `bool`.

```
bool a = true;
```

```
bool b = false;
```

Por ejemplo, en el siguiente programa la salida será “Los valores ingresados son iguales? `true` o `false`”, dependiendo de qué datos ingrese el usuario.

```
// declaro y leo por consola
int a,b;
cin >> a >> b;

// asigno el valor logico de la expresion a==b
bool sonIguales = (a==b);

// muestro por consola
cout << "los valores ingresados son iguales? " << sonIguales;
```

La expresión `(a==b)` es una expresión lógica cuyo valor de verdad dependerá de qué valores tengan asignadas ambas variables. Los paréntesis no son obligatorios, pero aportan algo de claridad.

#### 1.2.1.5. Tipo de dato nulo

El tipo `void` se utiliza para indicar que una función no retornará ningún valor. Esto lo veremos más adelante.

#### 1.2.1.6. Tipos de dato primitivos y tipos definidos por el programador

Los lenguajes de programación permiten que los programadores declaren sus propios tipos de dato. Por esto, agregaremos a la tabla de tipos que analizamos anteriormente una súper categoría que nos permitirá diferenciar entre tipos primitivos y tipos de dato definidos por el programador.

Los tipos de dato primitivos son parte del lenguaje. En cambio, para declarar nuevos tipos de dato utilizaremos recursos provistos por el lenguaje de programación.

En general, existen tres mecanismos a través de los cuales un programador puede declarar sus propios tipos de dato:

1. La palabra `typedef`.
2. Las estructuras (`struct`).
3. Las clases (`class`), de Programación Orientada a Objetos (POO).

Por ejemplo, `typedef` permite asignar una especie de alias a un tipo de dato ya existente; tal como lo observamos en el siguiente código, donde usamos `typedef` para declarar el tipo de dato `Entero`.

```
// ahora Entero es sinonimo de int
typedef int Entero;

int main()
{
    // declaro tres enteros
    Entero a,b,c;

    // :

    return 0;
}
```

Las estructuras (`struct`) se estudiarán más adelante, mientras que las clases de POO exceden el alcance de este curso. Pero sí mencionaremos que, en C++, `string` es una clase; no es un tipo de dato primitivo.

### 1.2.2. Alcance de una variable

Como ya mencionamos, los bloques de código se delimitan mediante llaves. Muy probablemente, un bloque de código pueda incluir a otros bloques, los cuales también podrían incluir más bloques de código.

Cuando declaramos una variable, ésta sólo existirá dentro del bloque de código en el que fue declarada, y también en todos sus subbloques.

En el siguiente ejemplo, vemos que el bloque de la función `main` incluye al bloque del `while`, que incluye los bloques del `if` y `else`. Y dentro de estos bloques de código declaramos algunas variables.

```
int a;

int main()
{
    int b;
    while( ... )
    {
        int c;
        if( ... )
        {
            int d;
        }
        else
        {
            int e;
        }
    }

    return 0;
}
```

La variable `b` es válida desde el inicio y hasta el final de la función `main`. Podemos utilizarla en cualquier parte del programa (dentro del `while`, `if` y `else`).

La variable `c` está declarada dentro del `while` por lo que sólo podremos utilizarla allí y dentro de los subbloques del `if` y `else`.

`d` y `e` sólo podrán ser utilizadas en el `if` y `else` respectivamente. Esto quiere decir que si dentro del `else` intentamos hacer `cout<<d`, tendremos un error de compilación indicando que la variable `d` no está declarada; no existe.

Finalmente, la variable `a` es una *variable global*. Puede ser usada dentro de cualquier bloque de código. Sin embargo, el uso de variables globales es **inadmisible**, por considerarse una **pésima práctica de programación**.

### 1.2.3. Metodología Top-Down

Comenzamos el capítulo analizando el algoritmo que resuelve el problema de *cruzar una calle*. Ya hemos visto que, si no se aproximan autos, entonces, efectivamente podemos cruzar. Pero *cruzar* implica *bajar el cordón, caminar hacia el otro cordón y subir el cordón de la vereda de enfrente*.

*Bajar el cordón* también es un algoritmo en sí mismo, que se compone de: *levantar un pie, adelantarlo, bajarlo, levantar el otro pie, adelantarlo y bajarlo*. Incluso, *levantar un pie* también es un algoritmo que podría implicar: *acelerar el pulso cardíaco, irrigar los músculos de la pierna, contraerlos*, etcétera.

La metodología *Top-Down* se emplea para dividir un problema en varios subproblemas más pequeños y específicos, los cuales también pueden ser divididos en subproblemas, y así sucesivamente hasta que no se justifique seguir dividiendo.

#### 1.2.3.1. Funciones

En C++ los programas se implementan como funciones. De hecho, ya mencionamos la función `main` que implementa el programa o algoritmo principal.

A medida que los algoritmos se vayan tornando más complejos indefectiblemente tendremos que dividirlos en algoritmos más pequeños y específicos, que también implementaremos como funciones.

Una función es un subprograma que, a partir de un conjunto de parámetros, realiza una tarea específica y devuelve (o retorna) un valor.

Por ejemplo: queremos desarrollar un programa donde el usuario ingresará una fecha, y validaremos que los datos ingresados sean consistentes. Es decir, que no ingrese: *35 de enero*, o *29 de febrero* de un año no bisiesto.

Veamos cómo, en el programa principal (función `main`), podemos resolver la interacción con el usuario y abstraernos de la lógica necesaria para validar la fecha. Esto es posible porque invocamos a la función `validarFecha`.

```
int main()
{
    // el usuario ingresa la fecha
    int dia,mes,anio;
```

```

cout << "Ingrese dia, mes y anio:";
cin >> dia >> mes >> anio;

// validamos la fecha ingresada
bool fechaOk = validarFecha(dia,mes,anio);

// si no es correcta mostramos un error
if( !fechaOk )
{
    cout << "La fecha ingresada es incorrecta";
}
else
{
    cout << "Todo bien :-)";
}

return 0;
}

```

La función `validarFecha` recibe tres parámetros: día, mes y año, y retorna un valor lógico indicando si los parámetros recibidos componen una fecha válida o no.

Si nos pudiésemos abstraer de cuántos días tiene el mes, `validarFecha` resultará relativamente simple. Por eso, nos apoyaremos en la función `diasDelMes` que nos permitirá avanzar sobre la solución de nuestro problema actual.

```

bool validarFecha(int dia,int mes,int anio)
{
    // cuantos dias tiene el mes
    int maxDias = diasDelMes(mes,anio);

    // si se cumplen las condiciones, la fecha es correcta
    bool ret;
    if( (dia>0 && dia<=maxDias) &&
        (mes>0 && mes<=12) && (anio>=0) )
    {
        ret = true;
    }
    else
    {

```

```

        ret = false;
    }

    return ret;
}

```

El próximo paso será resolver la función `diasDelMes`, que consiste en preguntar si se trata de un mes de 30 o de 31 días. Salvo que se trate del mes 2 (febrero), en cuyo caso la cantidad de días dependerá de que el año sea o no bisiesto.

La función `esAnioBisiesto` nos ayudará a abstraernos de este problema.

```

int diasDelMes(int mes,int anio)
{
    // en principio son 31 dias
    int dias=31;

    // si es uno de los meses de 30 dias
    if( mes==4 || mes==6 || mes==9 || mes==11 )
    {
        dias=30;
    }
    else
    {
        if( mes==2 ) // es febrero???
        {
            // si el anio es bisiesto entonces son 29 dias
            if( esAnioBisiesto(anio) )
            {
                dias=29;
            }
            else
            {
                dias=28;
            }
        }
    }

    return dias;
}

```

Finalmente, tenemos que desarrollar la función `esAnioBisiesto`. Aunque el cálculo que determina si un año es bisiesto sea un poco más complejo, aquí aceptaremos que un año es bisiesto si es divisible por 4, o por 400 pero no por 100.

```
bool esAnioBisiesto(int anio)
{
    bool esBisiesto = false;

    // es divisible por 4?
    if( anio%4==0 )
    {
        esBisiesto=true;
    }
    else
    {
        if( anio%400==0 && anio%100!=0 )
        {
            esBisiesto=true;
        }
    }

    return esBisiesto;
}
```

### 1.2.3.2. Prototipo de una función

El prototipo de una función es su encabezado: qué parámetros recibe, cuáles son sus tipos de dato, el nombre de la función, y cuál es el tipo de dato del valor de retorno. Por ejemplo, los prototipos de las funciones que utilizamos en el ejemplo anterior para validar que el usuario haya ingresado una fecha correcta son:

```
bool validarFecha(int,int,int);
int diasDelMes(int,int);
bool esAnioBisiesto(int);
```

No es necesario mencionar los nombres de los parámetros; sólo interesa la posición que ocupan y su tipo de dato. Aun así, también es correcto incluir los nombres de los parámetros, lo cual resulta muy útil si acompañamos al prototipo de la función con algunas líneas de documentación. Como en el siguiente caso:

```
// Esta funcion retorna true o false segun determine
// que dia, mes y anio componen una fecha valida o no
bool validarFecha(int dia,int mes,int anio);
```

### 1.2.3.3. Compilar un programa que invoca funciones

El código fuente de una función debe estar ubicado antes de su invocación. Esto quiere decir que, si en el programa principal invocamos a `validarFecha`, su código fuente debe estar colocado arriba del código de la función `main`.

Si en `validarFecha` invocamos a `diasDelMes`, el código fuente de esta última debe colocarse arriba del código de la primera, y así sucesivamente.

En el siguiente ejemplo vemos el desarrollo de la función `f`, y seguidamente, la función `main` que la invoca.

```
int f() // codigo de la funcion f
{
    // ...
}

int main() // programa principal
{
    f();
    // ...
}
```

Otra opción sería describir el prototipo de la función antes de su invocación; y luego, en cualquier parte del código fuente, desarrollarla.

```
// prototipo de f
int f();

// programa que invoca a f
int main()
{
    f();
    // ...
}
```

```
// desarrollo completo de la funcion f
int f()
{
    // ...
}
```

Como un programa puede estructurarse en cientos o miles de funciones, tener todo el código fuente en el mismo archivo puede tornarse muy complejo. Este problema lo abordaremos más adelante.

#### 1.2.3.4. Reusabilidad del código

Las funciones no sólo sirven para estructurar un programa, también permiten reutilizar diferentes porciones del código fuente.

Tomaremos como ejemplo la función `esAnioBisiesto`, que ya resolvimos y utilizamos para validar una fecha. Aceptaremos que un año es bisiesto cuando es divisible por 4, o por 400 pero no por 100.

Recordemos el código de la primera versión de `esAnioBisiesto`.

```
bool esAnioBisiesto(int anio)
{
    bool esBisiesto = false;

    // es divisible por 4?
    if( anio%4==0 )
    {
        esBisiesto=true;
    }
    else
    {
        if( anio%400==0 && anio%100!=0 )
        {
            esBisiesto=true;
        }
    }

    return esBisiesto;
}
```

```
}
```

Ahora veremos una nueva versión de `esAnioBisiesto` que invoca a la función `esDivisiblePor` para determinar si el año es o no divisible por 4, 100 y 400.

Esta nueva versión resulta ser algo más legible que la anterior porque no expone las operaciones del tipo `anio%4==0`, que podrían resultar confusas.

```
bool esAnioBisiesto(int anio)
{
    bool esBisiesto = false;

    if( esDivisiblePor(anio,4) )
    {
        esBisiesto=true;
    }
    else
    {
        if(esDivisiblePor(anio,400) &&
           !esDivisiblePor(anio,100))
        {
            esBisiesto=true;
        }
    }

    return esBisiesto;
}
```

Incluso, podemos reordenar el código para que se apegue tanto como sea posible a la definición de *año bisiesto*.

```
bool esAnioBisiesto(int anio)
{
    bool esBisiesto = false;

    // es divisible por 4, o por 400 y no por 100
    if(esDivisiblePor(anio,4) or
       esDivisiblePor(anio,400) and
       !esDivisiblePor(anio,100) )
    {
```

```

        esBisiesto=true;
    }

    return esBisiesto;
}

```

Todavía podemos mejorar el código un poco más. La siguiente expresión lógica tiene valor de verdad true o false.

`esDivisiblePor(4) or esDivisiblePor(100) and !esDivisiblePor(100)`

Así que `esAnioBisiesto` podría limitarse a retornar dicho valor de verdad.

```

bool esAnioBisiesto(int anio)
{
    // es divisible por 4 o por 400 y no por 100
    return esDivisiblePor(anio,4) or
           (esDivisiblePor(anio,100) and
            !esDivisiblePor(anio,100));
}

```

Ahora, desarrollaremos la función `esDivisiblePor` cuyo código es trivial.

```

bool esDivisiblePor(int a,int b)
{
    return a%b==0; // si al dividir a por b el resto es 0 ...
}

```

Podríamos reutilizar `esDivisiblePor` para resolver otras funciones, como por ejemplo `esPar` y `esImpar`.

```

bool esPar(int n)
{
    return esDivisiblePor(n,2);
}

```

```

bool esImpar(int n)
{
    return !esDivisiblePor(n,2);
}

```

Con `esDivisiblePor` también podríamos implementar `esMultiploDe`.

```
bool esMultiploDe(int a,int b)
{
    return esDivisiblePor(a,b);
}
```

Hay que subrayar que cuanto más pequeñas son las piezas de código que desarrollamos, mayores serán las posibilidades de que las podamos reutilizar.

#### 1.2.3.5. Legibilidad del código fuente

Las funciones ayudan a incrementar la *legibilidad*; por eso, es deseable que el código fuente de los programas sea fácil de comprender. La legibilidad favorece la *mantenibilidad*, porque ante la ocurrencia de un error resultará más simple identificar dónde se produjo y así solucionarlo.

#### 1.2.3.6. Bibliotecas de funciones

El código de las funciones debe estar colocado en archivos independientes. Esto permite que una misma función pueda ser invocada por diferentes programas.

Cuando desarrollamos funciones reutilizables y las colocamos en archivos independientes construimos nuestra propia biblioteca de funciones. Asimismo, los lenguajes de programación proveen sus propias bibliotecas.

Por ejemplo: las funciones `log` y `sqrt` que calculan, respectivamente, el logaritmo natural y la raíz cuadrada del argumento que reciben cómo parámetro son parte de la biblioteca `math.h` de C++. Por el momento, no profundizaremos sobre cómo construir una biblioteca de funciones, pues este tema lo abordaremos en detalle a partir del siguiente capítulo.

#### 1.2.3.7. Convención de nombres de funciones

Por convención, los nombres de las funciones se escriben en minúscula. Si el nombre se compone por dos o más palabras, cada inicial (excepto la primera) debe colocarse en mayúscula. Algunos ejemplos son:

```
double calcularPromedio(){ //... }
int diasDelMes(){ //... }
void mostrarMensaje(){ //... }
```

### 1.2.3.8. Parámetros y argumentos

Llamamos argumentos a los valores que le pasamos a una función. Por su parte, la función recibe esos valores como parámetros.

En la siguiente imagen, invocamos a la función `validarFecha` y le pasamos las variables `dia`, `mes` y `anio` como argumentos. Desde el punto de vista de la función, ésta recibe sus valores en los parámetros `d`, `m` y `a`.

```
// :
bool fechaOk = validarFecha(dia,mes,anio)
// :
bool validarFecha(int d,int m,int a)
{
    // ...
}
```

Figura 1.19. Argumentos y parámetros

Los nombres de los argumentos no tienen por qué coincidir con los nombres de los parámetros. Lo que sí debe coincidir es la posición.

### 1.2.3.9. Parámetros por valor y referencia

El parámetro recibe una copia del valor del argumento. Por esto, si en la función modificamos el valor de algún parámetro, esa modificación sólo tendrá alcance local y no trascenderá más allá de la misma función.

En el siguiente ejemplo, a la derecha, vemos que la función `permutar` recibe dos parámetros y permuta sus valores. En el programa de la izquierda leemos dos valores por consola; los permutamos y mostramos los valores. ¿Permutados?

```
int main()
{
    int x,y;
    cin >> x >> y;

    // permuto los valores
    permutar(x,y);

    cout << x << y << endl;
    return 0;
}
```

```
void permutar(int a,int b)
{
    int aux = a;
    a = b;
    b = aux;
    return;
}
```

Si el usuario ingresa 5 y 10 (para  $x$  e  $y$  respectivamente) se espera que la salida del programa sea 10 y 5, asumiendo que al invocar a la función `permutar` sus valores fueron permutados.

Sin embargo, dado que la función trabajó con copias de  $x$  e  $y$  contenidas en  $a$  y  $b$ , luego de finalizar su ejecución,  $x$  e  $y$  no resultarán modificados.

Para que una función pueda modificar los valores de los argumentos que recibe como parámetros, debe recibirlos por *referencia*. Para esto, alcanza con anteponer el símbolo `&` (*ampersand*) al nombre del parámetro que la función necesita modificar.

```
void permutar(int& a,int& b)
{
    int aux = a;
    a = b;
    b = aux;
    return;
}
```

Ahora sí podemos decir que  $a$  y  $b$  son  $x$  e  $y$ . Las modificaciones que hagamos sobre estos parámetros las estaremos haciendo a sus respectivos argumentos.

Observemos que la función `permutar` no retorna ningún valor. Por esta razón, el tipo de dato de su valor de retorno es `void` (nulo). Esto no guarda ninguna relación con el hecho de que la función reciba parámetros por valor o referencia.

### 1.2.3.10. Variables locales

Las variables que declaramos dentro de una función son variables locales y existen dentro de la función, y mientras dure su ejecución.

En la función `permutar`, por ejemplo, la variable `aux` es una variable local. Los parámetros `a` y `b`, independientemente de que se reciban por valor o referencia, también son variables locales. Todas dejarán de existir ni bien finalice la invocación.

Veamos el siguiente caso: en el programa principal (función `main`) declaramos la variable `a` y le asignamos el valor 5. Luego, invocamos a la función `f` que declara otra variable `a` y le asigna el valor 10.

La variable `a` de la función `main` no tiene nada que ver con la variable `a` de la función `f`. Por eso, cuando en la función `f` mostramos el valor de `a` la salida será 10. Luego, al finalizar la función, retornamos al programa principal y mostramos el valor de la variable `a`. En este caso, será 5, ya que su valor nunca fue alterado.

```
int main()
{
    int a = 5;

    // invoco a f
    f();

    cout << "En main: " << a;

    return 0;
}
```

```
void f()
{
    int a = 10;
    cout << "En la funcion: " << a;

    return;
}
```

### 1.2.4. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

## 1.3. Lección 3

Durante esta lección, aprenderemos a usar estructuras (`struct`) para diseñar nuevos tipos de dato creados por nosotros mismos, que nos ayudan a representar elementos de la vida real y ocultar la complejidad de sus operaciones relacionadas.

### 1.3.1. Estructuras

Las estructuras permiten crear nuevos tipos agrupando datos que tienen coherencia entre sí. Como ejemplo, veremos la estructura `Fecha` que agrupa tres datos: día, mes y año.

La fecha se compone por un día, mes y año. Estos datos son coherentes entre sí, ya que `día` representa un día del mes, y `mes` representa un mes del año `anio`. Los tres datos identifican una fecha.



Lección 3

```
struct Fecha
{
    int dia;
    int mes;
    int anio;
};
```

Luego de definir una estructura, queda establecido un nuevo tipo de dato que podremos utilizar para declarar variables.

```
// declaro una fecha
Fecha f1;
f1.dia = 2;
f1.mes = 10;
f1.anio = 1970;

// declaro otra fecha
Fecha f2;
f2.dia = 15;
f2.mes = 12;
f2.anio = 1985;
```

Las estructuras agrupan datos que llamaremos *campos*. La estructura `Fecha` tiene tres campos: `dia`, `mes` y `anio`.

Para simplificar, a las variables cuyo tipo de dato es una estructura también podremos llamarlas *estructuras*. Para asignar valor a los campos de una estructura utilizamos el operador `.` (punto) como separador. Por ejemplo:

```
Fecha f1;    // declaro la variable f1 tipo Fecha
f1.mes = 10; // asigno 10 al campo mes
```

Gráficamente, representamos a la estructura `Fecha` del siguiente modo:

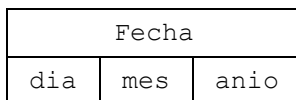


Figura 1.20. Representación de la estructura `Fecha`

#### 1.3.1.1. Inicialización de una estructura

Cuando asignamos valor inicial a los campos de una estructura decimos que la *inicializamos*. Esto podemos hacerlo de tres modos diferentes:

1. Asignando los valores de sus campos, uno por uno.
2. Asignando el conjunto completo de los valores de sus campos.
3. Creando una función de inicialización (lo veremos más adelante).

En la siguiente tabla, a la izquierda, asignamos uno a uno los valores de los campos de una variable tipo `Fecha`. A la derecha, declaramos la variable y le asignamos el conjunto completo de valores que queremos que sus campos tomen.

Los dos modos de inicialización son correctos; probablemente el segundo sea más práctico que el primero.

| Asignando campo por campo                                 | Asignando el conjunto de valores  |
|-----------------------------------------------------------|-----------------------------------|
| <pre>Fecha f; f.dia = 2; f.mes = 10; f.anio = 1970;</pre> | <pre>Fecha f = {2,10,1970};</pre> |

Tabla 1.11. Inicialización de una estructura.

### 1.3.1.2. Convención de nombres de estructuras

Por convención, los nombres de las estructuras deben escribirse en minúscula, a excepción de la primera letra, que deberá colocarse en mayúscula. Si el nombre se compone de dos o más palabras, cada inicial debe estar en mayúscula.

Algunos ejemplos son:

```
struct Persona
{
    // ...
};

struct CuentaCorriente
{
    // ...
};
```

### 1.3.1.3. Función de inicialización de una estructura

La función de inicialización de una estructura es una función creada con el único objetivo de asignar valores iniciales a los campos de la estructura. Por convención, esta función debe tener el mismo nombre que la estructura para la cual fue creada, aunque comenzando con minúscula. De esa forma, se respeta la convención de nomenclatura establecida para funciones.

A continuación, veremos dos modos válidos de implementar la función de inicialización (o *init function*) para la estructura `Fecha`.

#### Implementación 1

```
Fecha fecha(int d,int m,int a)
{
    Fecha f;
    f.dia = d;
    f.mes = m;
    f.anio = a;
    return f;
}
```

#### Implementación 2

```
Fecha fecha(int d,int m,int a)
{
    return {d,m,a};
}
```

Al disponer de la función de inicialización, podemos crear e inicializar muy fácilmente variables tipo Fecha.

```
// creamos dos fechas
Fecha f1 = fecha(2,10,1970);
Fecha f2 = fecha(21,3,1992);
```

#### 1.3.1.4. Estructuras anidadas

Los campos de una estructura pueden ser de cualquier tipo, incluso una estructura. Veamos la estructura `Direccion`, con su correspondiente función de inicialización. A continuación, la estructura `Persona` con los campos `fechaNacimiento` (tipo Fecha) y `direccion` (tipo `Direccion`).

```
// estructura direccion
struct Direccion
{
    string calle;
    int numero;
    int piso;
    char departamento;
    string codigoPostal; // ej: C1432BQZ
};

Direccion direccion(string c,int n,int p,char d,string cp)
{
    return {c,n,p,d,cp};
};
```

```
// estructura Persona
struct Persona
{
    string nombre;
    int dni;
    Fecha fechaNacimiento; // estructura anidada
    Direccion direccion;   // estructura anidada
};
```

Como los campos de una estructura pueden ser estructuras en sí mismos, el operador . (punto) nos permitirá hacer referencia a los campos de los campos de la estructura. Para darles valor inicial podemos hacerlo directamente, tal como vemos a la izquierda, usar el operador { } (llave) como vemos a la derecha, o hacerlo a través de sus correspondientes funciones de inicialización, como vemos más abajo.

```
Persona p;
p.nombre = "Pablo";
p.fechaNacimiento.dia = 2;
```

```
Persona p;
p.nombre = "Pablo";
p.fechaNacimiento={2,10,1970};
```

```
Persona p;
p.nombre = "Pablo";
p.fechaNacimiento = fecha(2,10,1970);
p.direccion = direccion("El Alamo",25,1,'B',"1432");
```

Por supuesto que el modo más prolijo de crear una variable tipo `Persona` es a través de su función de inicialización, cuyo código veremos a continuación.

```
Persona persona(string n,int dni,Fecha fec,Direccion dir)
{
    return {n,dni,fec,dir};
}
```

Ahora, podemos crear una variable tipo `Persona` de la siguiente manera:

```
Persona p = persona("Pablo"
                    ,23342124
                    ,fecha(3,12,1985)
                    ,direccion("El Alamo",25,1,'B',"1432"));
```

### 1.3.2. Tipo Abstracto de Dato (TAD)

*Tipo Abstracto de Dato* (en adelante TAD) es un concepto que nos permite crear tipos de dato para representar objetos (elementos) de la vida real, con el objetivo de abstraernos de su implementación y la complejidad asociada a sus operaciones. Por

ejemplo una fecha, un número fraccionario, o cualquier otro elemento de la vida cotidiana que necesitemos manipular en nuestros programas.

Un TAD se compone de dos partes: una estructura para guardar datos, y un conjunto de funciones diseñadas para manipular los datos de la estructura.

$$TAD = Datos + Funciones$$

Por convención, los nombres de las funciones llevarán como prefijo el nombre de la estructura (comenzando en minúscula), que a su vez será el nombre del TAD.

Para comprender la idea, comenzaremos analizando un programa que usa el TAD *Fecha*, asumiendo que dicho TAD ya está programado y listo para utilizar.

En el siguiente ejemplo, le pedimos al usuario que ingrese un día, mes y año; y con esos datos creamos una fecha. Luego, le pedimos que ingrese una cantidad de días, y sumamos dicha cantidad a la fecha anterior. Así, obtenemos una nueva fecha que mostraremos por consola.

```
int main()
{
    // pedimos un dia, mes y anio
    int d,m,a;
    cout << "Ingrese dia, mes y anio:";
    cin >> d >> m >> a;

    // creamos un objeto fecha con los datos ingresados
    Fecha f1 = fecha(d,m,a);
    cout << "Usted ingreso la fecha:" << fechaToString(f1);

    // pedimos que ingrese una cantidad de dias
    int n;
    cout << "Ingrese una cantidad de dias: ";
    cin >> n;

    // le sumamos n dias a la fecha anterior
    Fecha f2 = fechaAddDias(f1,n);

    // obtenemos una cadena que representa la fecha f2
    string s = fechaToString(f2);

    cout << "Dentro de "<< n << " dias sera: " << s;

    return 0;
}
```

En el código anterior aceptamos que disponemos de la estructura `Fecha` y las funciones `fecha` (función de inicialización), `fechaToString` y `fechaAddDias`.

Observemos que en el programa no fue necesario manipular ninguno de los campos de la estructura `Fecha`. Es más, ni siquiera sabemos si se trata de la misma estructura `Fecha` que estudiamos previamente, pues no tuvimos la necesidad de conocer su implementación.

Asumiendo que `Fecha` efectivamente es la estructura que declaramos más arriba, y `fecha` es su función de inicialización que previamente implementamos, veamos cómo implementar la función `fechaToString` para crear y retornar una cadena que sea la representación de la fecha contenida en el parámetro `f`.

```
string fechaToString(Fecha f)
{
    // retornamos dia/mes/año
    string s= to_string(f.dia)+"/"
              +to_string(f.mes)+"/"
              +to_string(f.año);
    return s;
}
```

La función `to_string` es una función de la biblioteca de C++, que podemos utilizar agregando la línea `#include<string>` en el encabezado del programa.

Por último, la implementación de `fechaAddDias` no es tan simple, pues hay que tener en cuenta que al sumarle `n` días a una fecha podría variar el mes (dependiendo de cuántos días tiene el mes); y al variar el mes, cambiar el año.

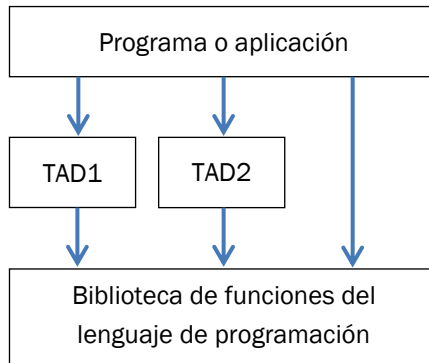
```
Fecha fechaAddDias(Fecha f, int n)
{
    Fecha ret;

    // :
    // que hacemos aqui... ???
    // :

    return ret;
}
```

### 1.3.2.1. Usuario del TAD

Un TAD es una herramienta que creamos y/o utilizamos para facilitar el desarrollo de nuestros programas, incrementando su legibilidad y mantenibilidad. Por esta razón, quien utiliza un TAD es un programador; no el usuario final del programa.



*Figura 1.21. Stack de programación. La aplicación usa TAD y funciones del lenguaje. Los TAD se programan usando las funciones del lenguaje de programación*

### 1.3.2.2. Inicialización de un TAD

Para evitar que el usuario tenga que manipular la implementación de la estructura, debemos proveer una función de inicialización cuyo nombre debe coincidir con el nombre de la estructura, aunque comenzando en minúscula.

### 1.3.2.3. Sobrecarga de funciones

Podemos escribir varias versiones de una misma función siempre y cuando todas tengan diferentes prototipos. Por ejemplo: la función de inicialización `fecha` permite crear fechas a partir de los atributos día, mes y año.

Probablemente podría interesarnos permitir que el usuario pueda crear fechas a partir de una cadena de caracteres, como por ejemplo: "02/10/1970".

Para esto tenemos que sobrecargar la función de inicialización `fecha`.

```
Fecha fecha(int d,int m,int a){ //... }
Fecha fecha(string s){ //... }
```

Ahora tenemos dos funciones llamadas `fecha`. La primera recibe tres enteros; la segunda, una cadena de caracteres. Cuando invoquemos a `fecha`, dependiendo de qué argumentos le pasemos, el compilador sabrá a qué versión de la función estamos invocando e invocará a la versión correcta.

```
// creamos una fecha a partir de tres enteros
Fecha f1 = fecha(2,10,1970);

// creamos una fecha a partir de una cadena
Fecha f2 = fecha("25/11/1996");
```

Cabe aclarar que podemos sobrecargar cualquier función, independientemente de que sea o no parte de un TAD. La sobrecarga de funciones no es una propiedad exclusiva de los TAD.

### 1.3.3. Excepciones y aserciones

#### 1.3.3.1. Introducción a las excepciones

Hasta ahora hemos asumido que las funciones sólo reciben datos válidos. Sin embargo, en la práctica esto no siempre será así. Es frecuente que una función reciba parámetros incorrectos o inconsistentes con los cuales no podrá realizar su tarea.

Por ejemplo, en el siguiente código invocamos a la función `factorial` y le pasamos como argumento un valor negativo.

```
long long n = factorial(-5);
```

#### 1.3.3.2. Código de error

Con los recursos que contamos hasta ahora, el único modo de hacerle saber al usuario que la función no ha podido realizar su tarea consiste en devolver un valor especial, comúnmente denominado *código de error*. Por ejemplo, podríamos decidir que la función `factorial` devuelva `-1` cuando reciba un valor negativo, y de este modo indicar que no existe un resultado válido para esa operación.

### 1.3.3.3. Excepciones

Las excepciones nos dan otra posibilidad. Permiten no devolver ningún valor de retorno y finalizar inmediatamente la ejecución de una función lanzando una excepción. De este modo, le haremos saber al usuario (programador que invoca la función) que la operación no pudo completarse y que la ejecución de la función fue abortada.

Decimos entonces que la función finalizó por excepción o, lo que es lo mismo, que lanzó una excepción.

### 1.3.3.4. Lanzando excepciones con throw

La instrucción `throw` genera y lanza una excepción dentro de una función.

```
long long factorial(int n)
{
    if( n<0 )
    {
        string mssg = "n (" + to_string(n) + ") debe ser >= 0";
        throw runtime_error(mssg);
    }

    // resto de la funcion
}
```

Para que el código anterior compile será necesario agregar las siguientes dos líneas al principio del archivo de código fuente.

```
#include <stdexcept>
using std::runtime_error;
```

Cuando se ejecuta `throw`, la función lanza una excepción y finaliza inmediatamente. Todo el código que se encuentre después de la instrucción `throw` ya no se ejecutará.

```
// valor negativo para que falle factorial
int n = -10;
```

```
cout << "Veamos cual es el factorial de " << n << endl;

// invocamos a factorial con un valor negativo
long long x = factorial(n);

// esto no se ejecuta
cout << "El factorial de: " << n << " es:" << x << endl;
```

La llamada a la función `factorial` pasándole un valor negativo provocará un error en tiempo de ejecución, y así lo veremos reflejado en la consola:

```
terminate called after throwing an instance
of 'std::runtime_error' what(): n (-10) debe ser >= 0
```

#### 1.3.3.5. Excepciones y programación APT (A Prueba de Tontos)

Al desarrollar una función o un programa podemos adoptar dos filosofías diferentes.

La primera consiste en asumir que quien utiliza nuestras funciones siempre proporcionará datos válidos. En este enfoque, la responsabilidad de evitar errores recae sobre quien utiliza el programa o invoca la función. A esta forma de programar la llamaremos *No A Prueba de Tontos* (No APT).

La segunda consiste en desconfiar de los datos recibidos y verificar que sean consistentes antes de utilizarlos. En este caso, la función intenta detectar usos incorrectos y actuar en consecuencia. A esta forma de programar la llamaremos *A Prueba de Tontos* (APT).

Ninguna de estas filosofías es intrínsecamente mejor que la otra. Sin embargo, la programación APT suele producir programas más robustos, más fáciles de depurar y menos propensos a comportamientos inesperados.

Las excepciones constituyen una herramienta muy útil para implementar programación APT, ya que permiten detectar errores e informar claramente qué condición impidió que una función completara su ejecución.

A continuación, veremos dos implementaciones de la función `factorial`: una desarrollada siguiendo la filosofía No APT y otra siguiendo la filosofía APT.

## Programación NO APT

```
long long factorial(int n)
{
    long long f=1;
    int i=2;
    while( i<=n )
    {
        f = f*i;
        i = i+1;
    }

    return f;
}
```

## Programación APT

```
long long factorial(int n)
{
    if( n<0 )
    {
        string mssg="Negativo";
        throw runtime_error(mssg);
    }

    long long f=1;
    int i=2;
    while( i<=n )
    {
        f = f*i;
        i = i+1;
    }

    return f;
}
```

Si invocamos a la versión No APT de `factorial` pasándole un valor negativo retornará 1, lo cual claramente es una inconsistencia. En cambio, si el valor negativo se lo pasamos a la versión APT, esta no retornará nada y finalizará por excepción.

### 1.3.3.6. Capturando excepciones con try-catch

Las excepciones pueden ser interceptadas mediante un bloque `try-catch`. Decimos que intentamos (`try`) ejecutar una porción de código que podría fallar. Si durante su ejecución se lanza una excepción, esta es capturada (`catch`) y podemos emprender una acción correctiva.

Cuando se lanza una excepción, la ejecución del bloque `try` se interrumpe y el programa continúa en el bloque `catch` correspondiente.

```
try
{
    cout << "Ingrese un valor para calcular su factorial:";

    int n;
    cin >> n;
```

```

    long long x = factorial(n);
    cout << "Resultado: " << x << endl;
}
catch(runtime_error e)
{
    // mostramos el mensaje de error de la excepción
    cout << "Ocurrió un error: " << e.what() << endl;
}

```

### 1.3.3.7. Aserciones y programación a prueba del programador tonto (APPT)

Podríamos ir un paso más allá y clasificar *quién* es el responsable de que nuestro código falle. Esto es: programar a prueba de *fallos del usuario final* o hacerlo a prueba de *fallos del programador* que invoca a nuestras funciones, o ambas cosas.

Cuando validamos datos de entrada o manejamos excepciones, nos estamos defendiendo principalmente de los imprevistos del mundo real y del usuario final. Sin embargo, de aquí en más, nos enfocaremos en programar A Prueba del Programador Tonto. Es decir, evitar que seamos nosotros mismos, u otros programadores, quienes usemos mal nuestras propias funciones durante la etapa de desarrollo.

### 1.3.3.8. Aserciones

La función de biblioteca `assert` nos permite *asegurar* que se cumple una determinada condición. Y si no es así, hará que el programa entero aborte su ejecución.

Implementación usando `throw`:

```

long long factorial(int n)
{
    if( n<0 )
    {
        string mssg="Negativo";
        throw runtime_error(mssg);
    }

    // codigo de la funcion
    // :
}

```

Implementación usando `assert`:

```

long long factorial(int n)
{
    assert(n>=0);

    // codigo de la funcion
    // :
}

```

La versión de `factorial` de la izquierda lanza una excepción si el valor del parámetro `n` es negativo. En este caso, el programador tendrá la opción de capturar la excepción y mostrarle al usuario final un mensaje de error amigable.

En cambio, la versión de la derecha usa `assert` para *garantizar* de que el valor de `n` es mayor o igual a cero. Y si así no fuera, la función (y toda la ejecución del programa) será abortada. Y no habrá una instancia posterior de `catch`, como sí la hay cuando usamos `throw`.

En este punto, el lector pensará que la función `assert` hace algo como esto:

```
void miPropioAssert(bool expr)
{
    if( !expr )
    {
        exit(1); // finalizo el programa completo
    }
}
```

Siendo así, ambas llamadas son equivalentes:

```
long long factorial(int n)
{
    assert(n>=0);

    // codigo de la funcion
    // :
}
```

```
long long factorial(int n)
{
    miPropioAssert(n>=0);

    // codigo de la funcion
    // :
}
```

Y, de hecho, lo son. Salvo que (y aquí es donde veremos la gran ventaja de `assert`), `assert` mostrará un mensaje de error en la consola indicando cuál es la condición que no se ha verificado y en qué línea del código ocurrió.

Esta información resulta invaluable para depurar nuestro código. Entonces, si corremos el siguiente programa que invoca a la función `factorial` pasándole un argumento negativo, podremos ver concretamente por qué falló, qué condición no se verificó y en qué línea ocurrió.

```

01. long long factorial(int n)
02. {
03.     assert(n >= 0);
04.
05.     // :
06. }
07.
08. int main()
09. {
10.     int n = -1;
11.     long long x = factorial(n); // n es negativo !!!
12.     cout << x << endl;
13.
14.     return 0;
15. }

```

El mensaje de error que aparecerá en la consola será similar a este:

```
Assertion failed: n >= 0, file main.cpp, line 3
```

#### 1.3.3.9. Agregando un mensaje descriptivo al assert

`assert` muestra por consola la expresión lógica que recibe como parámetro. Si esa expresión es muy compleja, el mensaje que aparecerá cuando falle podría resultar poco claro o difícil de interpretar.

Afortunadamente, podemos aprovechar una característica del lenguaje C (que también es válida en C++). En C los valores enteros (`int`) y los punteros (tema que aún no hemos estudiado) tienen valor de verdad. Un entero con valor 0 se interpreta como falso, mientras que cualquier valor distinto de 0 se interpreta como verdadero. Del mismo modo, un puntero con valor `NULL` se considera falso, y con cualquier otro valor será verdadero.

Las cadenas literales son punteros a carácter (`char*`), y como están almacenadas en memoria son, por definición, punteros válidos (no nulos). Su *valor de verdad* es `true`. Entonces, usaremos esto a nuestro favor, así:

```

long long factorial(int n)
{

```

```

    assert(n>=0 && "No existe factorial de nros. negativos");

    // codigo de la funcion
    // :
}

```

En este caso le pasamos al `assert` una expresión compuesta por dos expresiones lógicas simples, unidas por un `&&` (*and*):

```
n>=0 && "No existe factorial de nros. negativos"
```

La primera condición es la que queremos evaluar. La segunda (la cadena) es `true` por definición. Entonces, el valor de verdad de: `loQueSea && true` es el valor de verdad de `loQueSea`. Pero como la cadena ahora es parte de la expresión a evaluar, el mensaje que veremos en caso de error será:

```
Assertion failed: n >= 0 && No existe factorial de nros.
negativos, file main.cpp, line 3
```

### 1.3.4. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

## 1.4. ¿Qué sigue?

En el próximo capítulo trabajaremos fuertemente con cadenas de caracteres. Desarrollaremos varias funciones para manipular caracteres integrando así nuestra propia biblioteca de funciones; la cual utilizaremos para gestionar colecciones de datos. Es decir: dentro de una cadena guardaremos datos, separados uno del otro por un carácter separador. Luego, extenderemos este concepto para convertir los datos en objetos y, de este modo, crearemos una herramienta que nos permitirá administrar colecciones de objetos. La llamaremos TAD `Coll`.